

AI 工程化训练营 模块一

尹会生

层级名称	对应模块	支撑体系
I. 基础能力层 构建可复用的工程单元	模块一：大模型技术栈与Prompt工程 模块二：NLP与微调基础	-工程工具链： • LangChain / LlamaIndex（基础交互） • Autogen / CrewAI（Agent协作框架）
II. 数据与知识层 构建企业知识中枢	模块三：数据工程与知识增强	-工程工具链： • LangSmith / LangFuse（知识追踪与评估） -工程化维度： • 可观测性（日志/评估） • 安全性（数据合规）
III. 系统架构层 构建可扩展的智能体架构	模块四：智能客服系统架构设计 模块五：多Agent协作与通信机制 模块六：DSL语言设计与执行引擎 模块七：智能Agent高级能力构建	-工程工具链： • Autogen / CrewAI（多Agent框架） • LangChain（流程编排） -工程化维度： • 可扩展性（插件化/分布式） • 可维护性（DSL/热更新） • 稳定性（熔断/重试）
IV. 部署与运维层 构建生产级可靠性	模块八：模型部署与服务化 模块九：Python高性能编程与并发工程	-工程工具链： • Docker / Kubernetes（容器化部署） • Prometheus / Grafana（监控系统） -工程化维度： • 稳定性（监控/告警） • 性能优化（并发/异步） • 可观测性（指标采集） -工程生命周期： • 部署上线 → 监控优化
V. 业务应用层 实现商业价值闭环	模块十：企业级智能客服平台 模块十一：行业场景与产品设计	-工程化维度： • 所有五大维度均在此落地： 稳定性、可维护性、可扩展性、可观测性、安全性 -工程生命周期： • 全流程闭环： 需求分析 → 架构设计 开发实现 → 测试验证 部署上线 → 监控优化 迭代演进 → 价值闭环 -商业价值闭环： • 产品落地 • 客户满意度提升 • ROI 可衡量

目录

阶段	模块	目标
认知层	第1讲：什么是 AI 工程化？	宏观视角理解AI从实验到落地的关键挑战
能力层	第2讲：大模型调用与函数调用	掌握与LLM交互的基础能力
工具层	第3讲：LangChain 核心组件详解	学会使用工业级框架搭建流程
数据层	第4讲：LlamaIndex 与知识增强系统	解决“如何让AI知道企业内部信息”问题
控制层	第5讲：Prompt Engineering 高阶技巧	提升AI推理质量与可控性
架构层	第6讲：Agent 设计与多轮对话逻辑	构建有状态、可追踪的智能体
协同层	第7讲：Autogen 与多Agent协作机制	实现团队式AI协作
个性化层	第8讲：轻量微调方法比较	理解何时及如何定制模型行为
落地层	第9 讲：AI系统工程化实践（生产部署）	生产实践

01 什么是 AI 工程化？

 认知层 建立宏观视角，理解AI从实验到落地的关键挑战

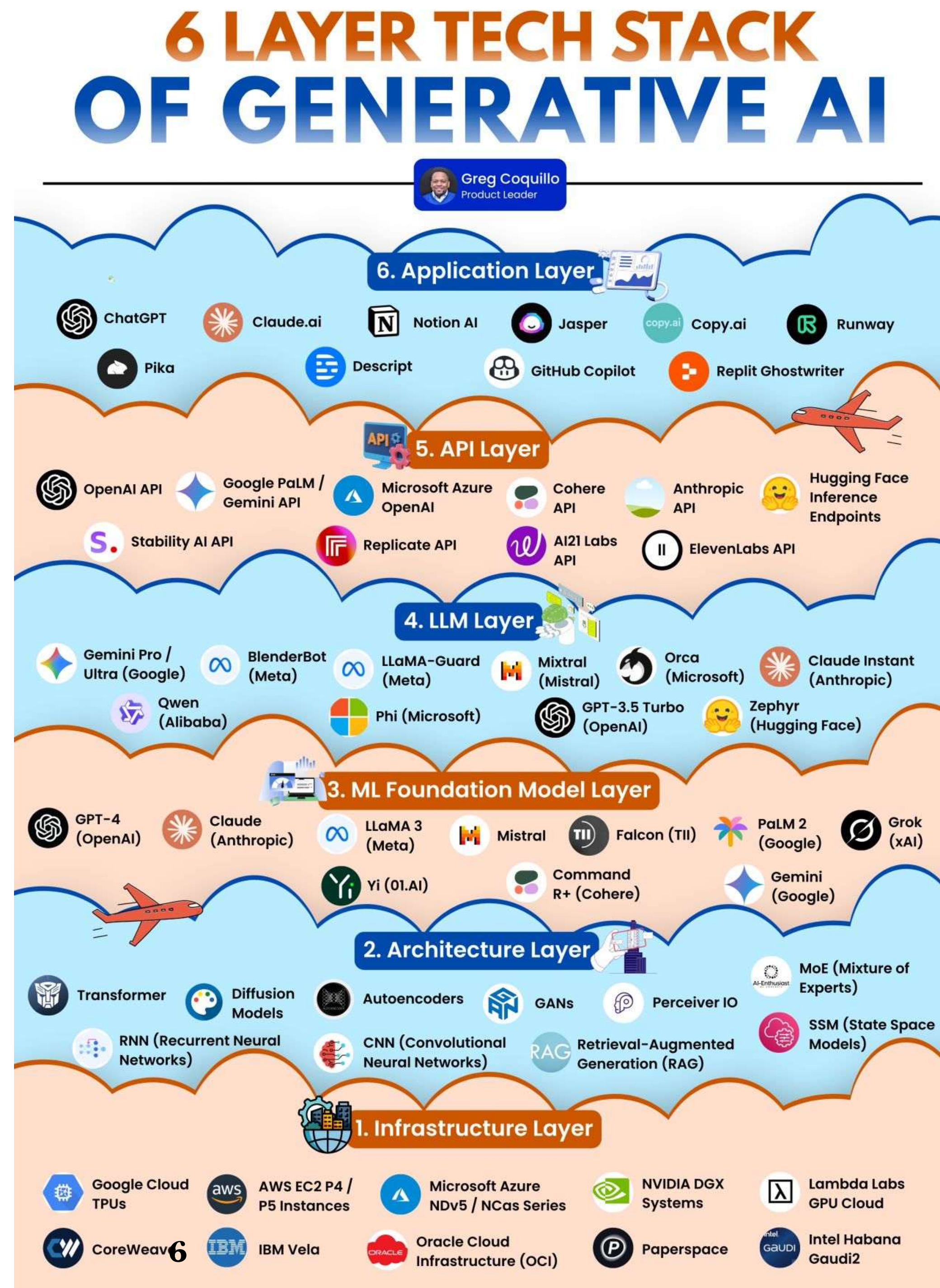
AI 工程化的定义

- 1、 将AI研究成果转化为稳定（可靠）、可维护（可控）、可扩展的生产系统
- 2、 分层架构（Layered Architecture）



AI 工程化的定义

从“OSI”模型到“TCP”“UDP”协议
把你知道的模型、软件、工具摆放到不同的
层级、不同的位置

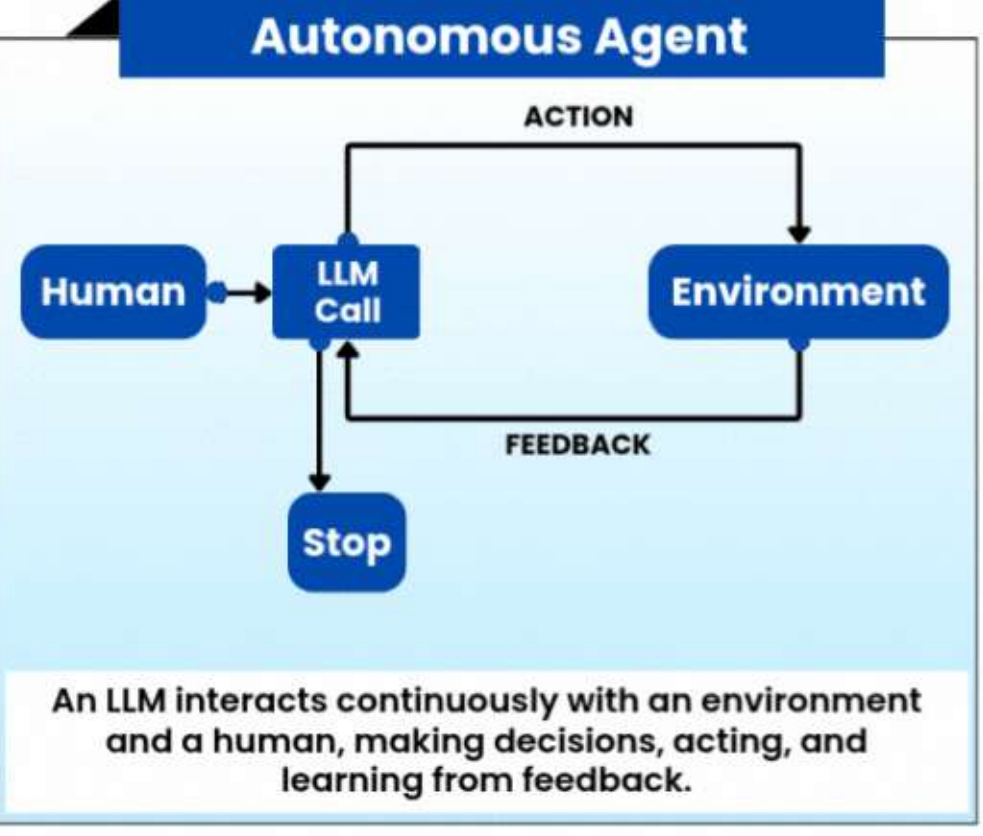
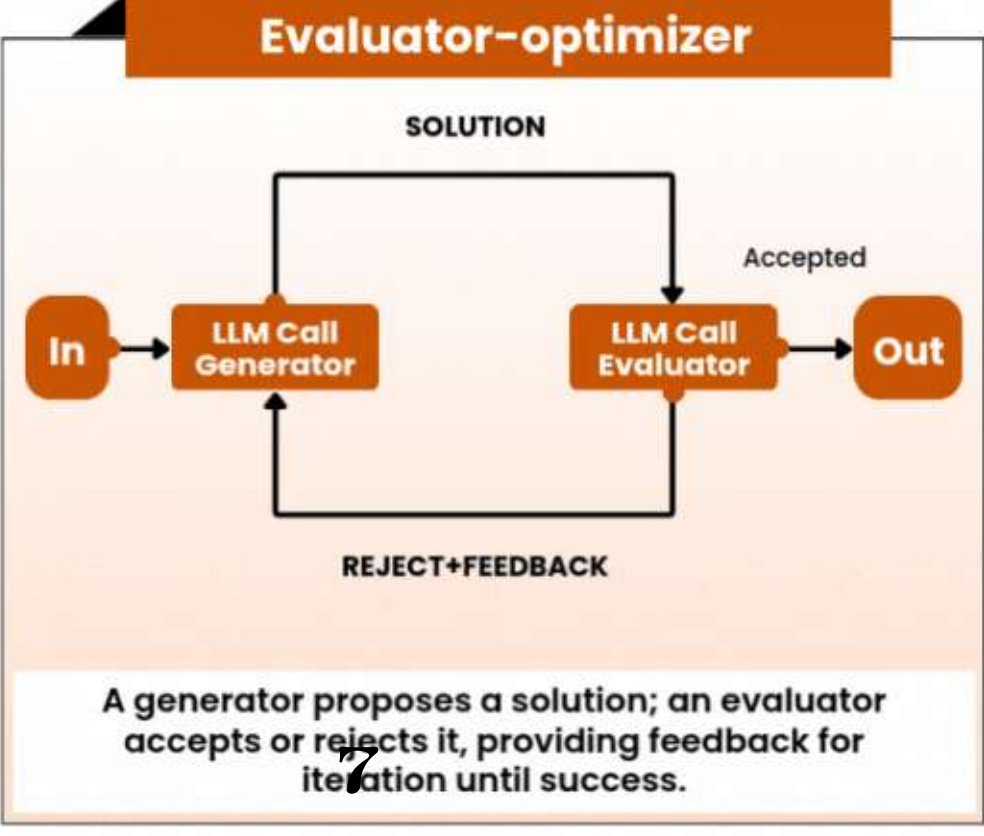
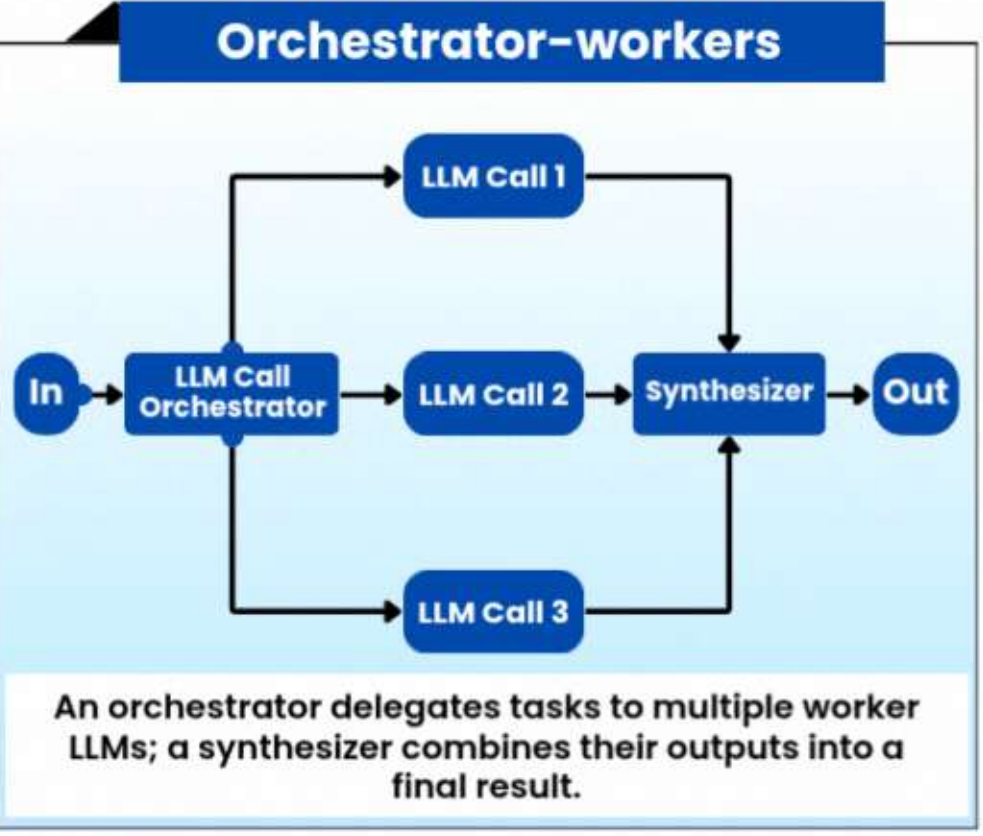
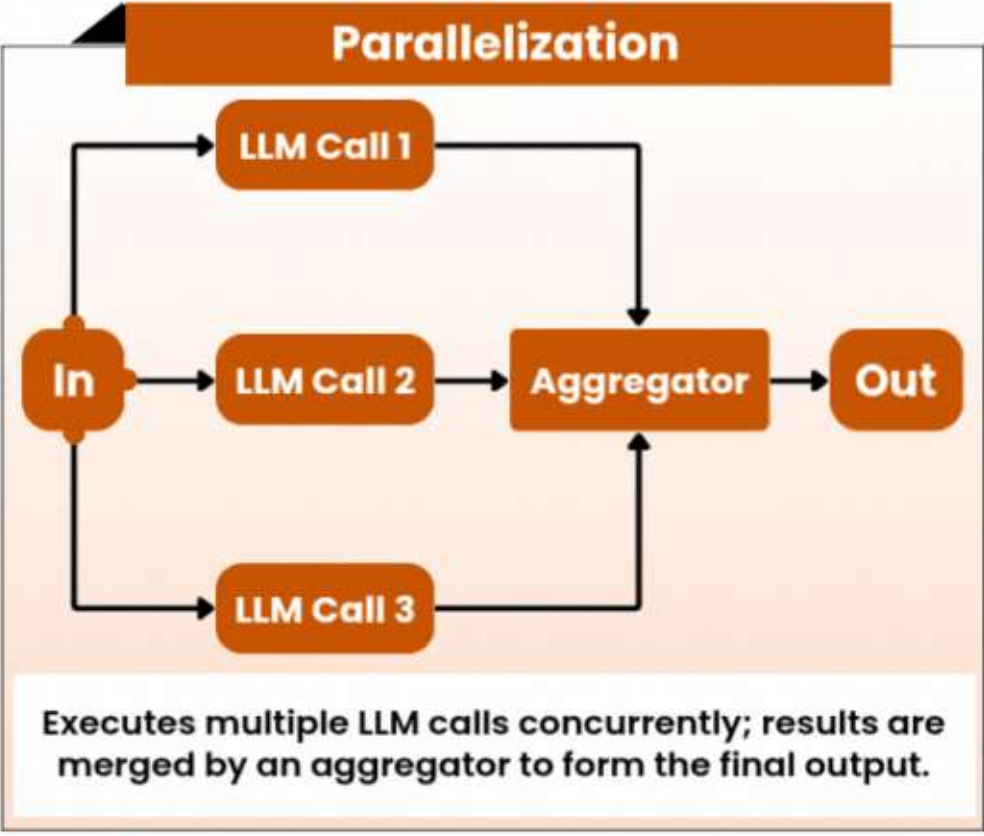
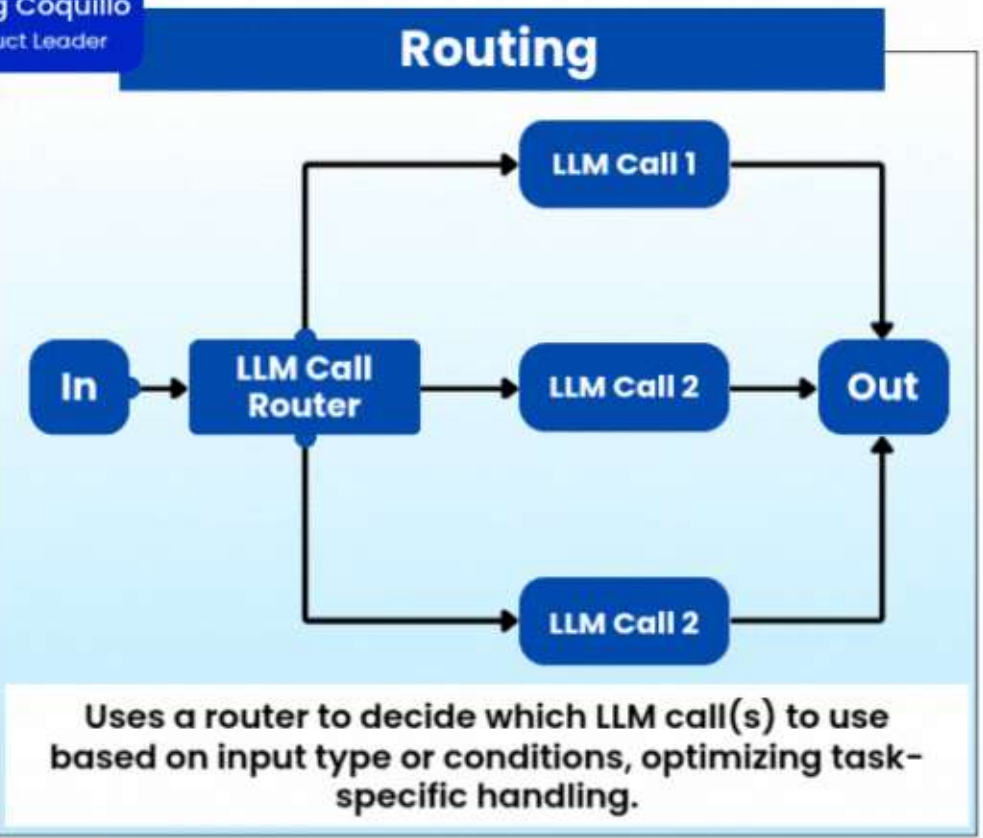
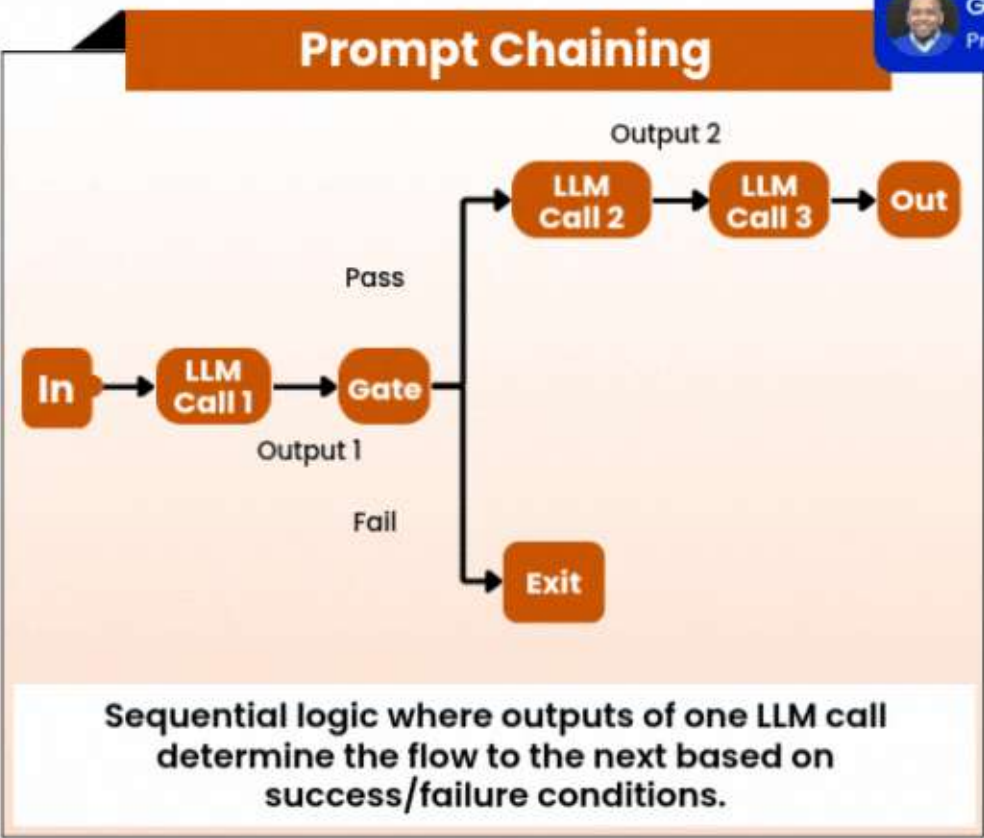


AI 工程化的定义

从 “TCP” 协议到 “poll”、“epoll” 模型

MASTERING LLM ARCHITECTURES 6 CORE ORCHESTRATION PATTERNS

Greg Coquillo
Product Leader



传统AI开发 vs. AI工程化开发

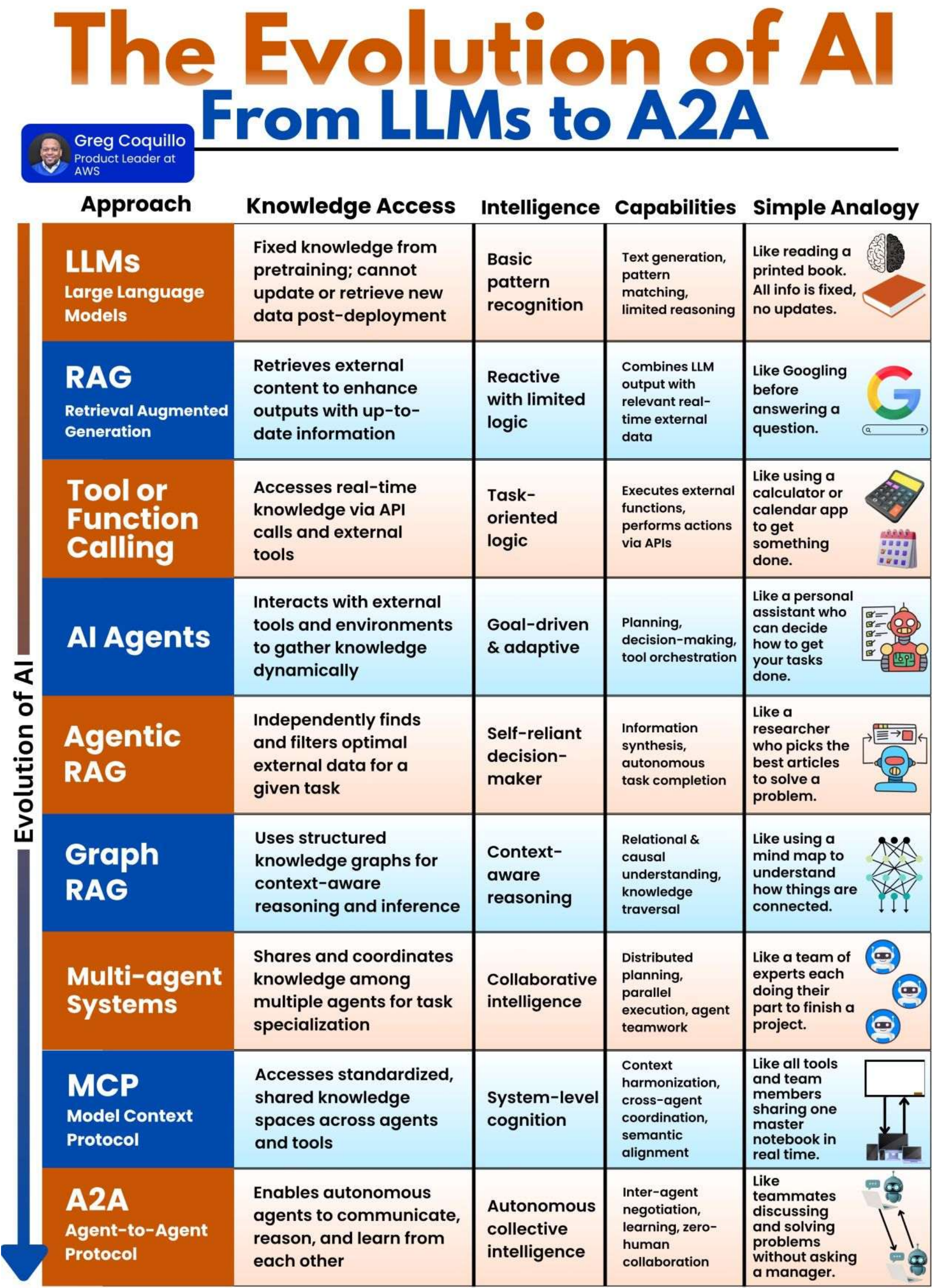
维度	传统AI开发（实验原型）	AI工程化开发（工业级系统）
调用方式	单次调用（One-shot Inference）	多轮交互（Multi-turn Orchestration）
Prompt管理	静态Prompt（Hard-coded）	动态流程控制（Context-aware, Adaptive）
模型使用	独立模型（Standalone LLM）	工具集成 + 多Agent协作（Tool-Augmented, Multi-Agent）
目标	验证想法、快速出Demo	可靠、可扩展、可维护的生产系统
系统边界	模型即系统	模型是组件之一，系统包含编排、监控、反馈等

实验原型 vs. 工业级系统

维度	实验原型	工业级系统
稳定性	偶尔失败可接受	高可用、容错、降级机制
可维护性	代码混乱、无文档	模块化、可配置、可回滚
可观测性	无日志、无监控	全链路追踪、日志、指标、告警
安全性	无权限控制	输入过滤、敏感词检测、权限校验
性能	延迟高、吞吐低	缓存、批处理、异步执行

典型痛点（核心挑战）

- 如何让AI调用数据库？
- 如何让AI分步骤解决问题？
- 如何让多个AI协同工作？



思考

1 设计问答助手 (Multi-Task QA Assistant)

用户提问：

“帮我查一下上个月销售额最高的三个产品，它们的客户评价如何？并生成一个PPT大纲。”

如何设计系统流程？

用什么技术栈实现？

思考

2 多Agent客服系统 (Multi-Agent Customer Service)

用户提问：

“用户：我的订单还没发货，已经三天了！”

选择哪种 Agent 模式？

划分为几个 Agent ？

02 大模型调用与函数调用

🔧 能力层 掌握与LLM交互的基础能力

主流大模型调用方式

1. OPENAI 方式

`https://platform.openai.com/docs/overview`

2. SDK 方式

`https://bailian.console.aliyun.com/?tab=doc#/doc`

3. HTTP 方式

`curl`

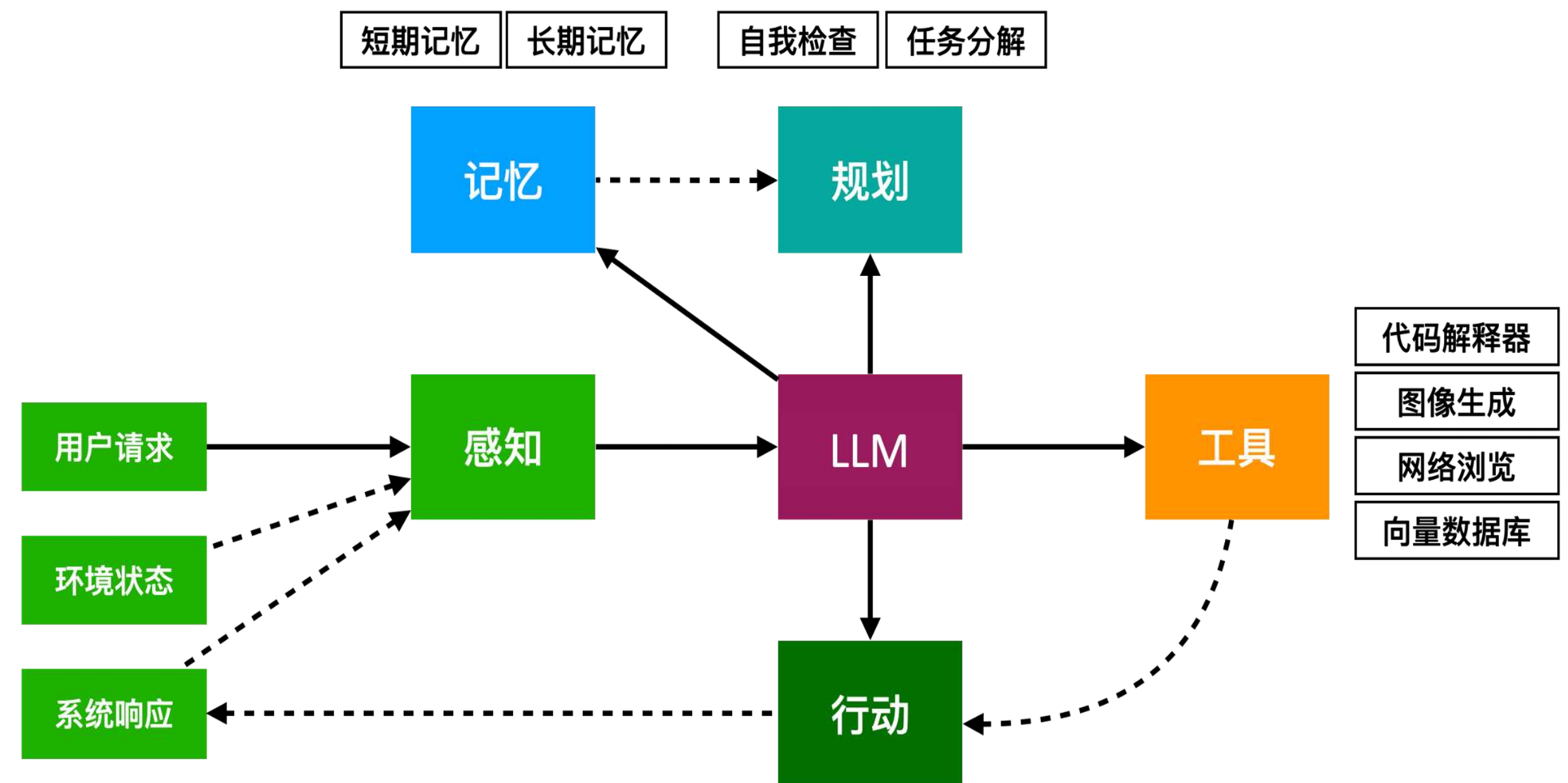
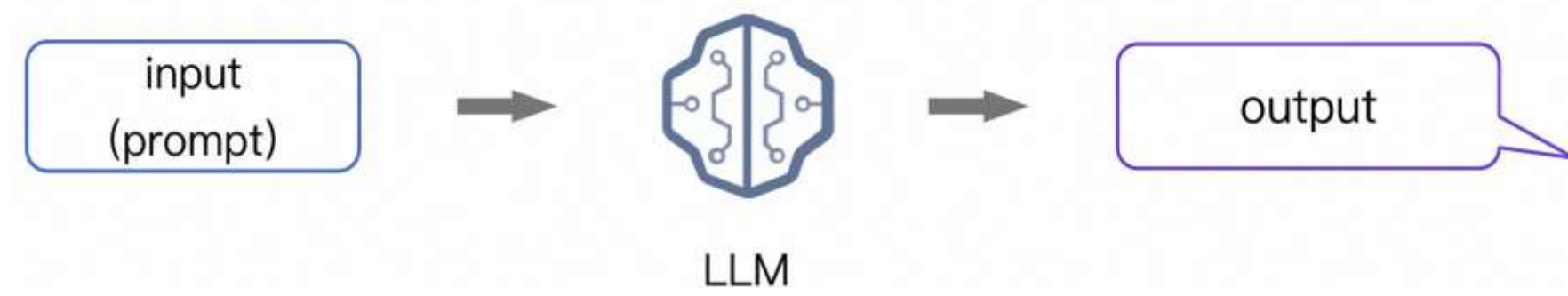
4. 借助低代码平台

如: cherry studio、coze、Dify等

5. 封装chat内容为 query () 函数

6. 别忘了跑单元测试!!!

从 Chat 到 Agent 的“软”技能和“硬”技能



OpenAI 实现工具的原理

OpenAI 的 Function Calling (现称为 Tool Calling) 是构建智能 Agent 的核心技术之一，它改变了传统 LLM 直接生成答案的模式，转而让模型“思考”是否需要调用外部工具来完成任

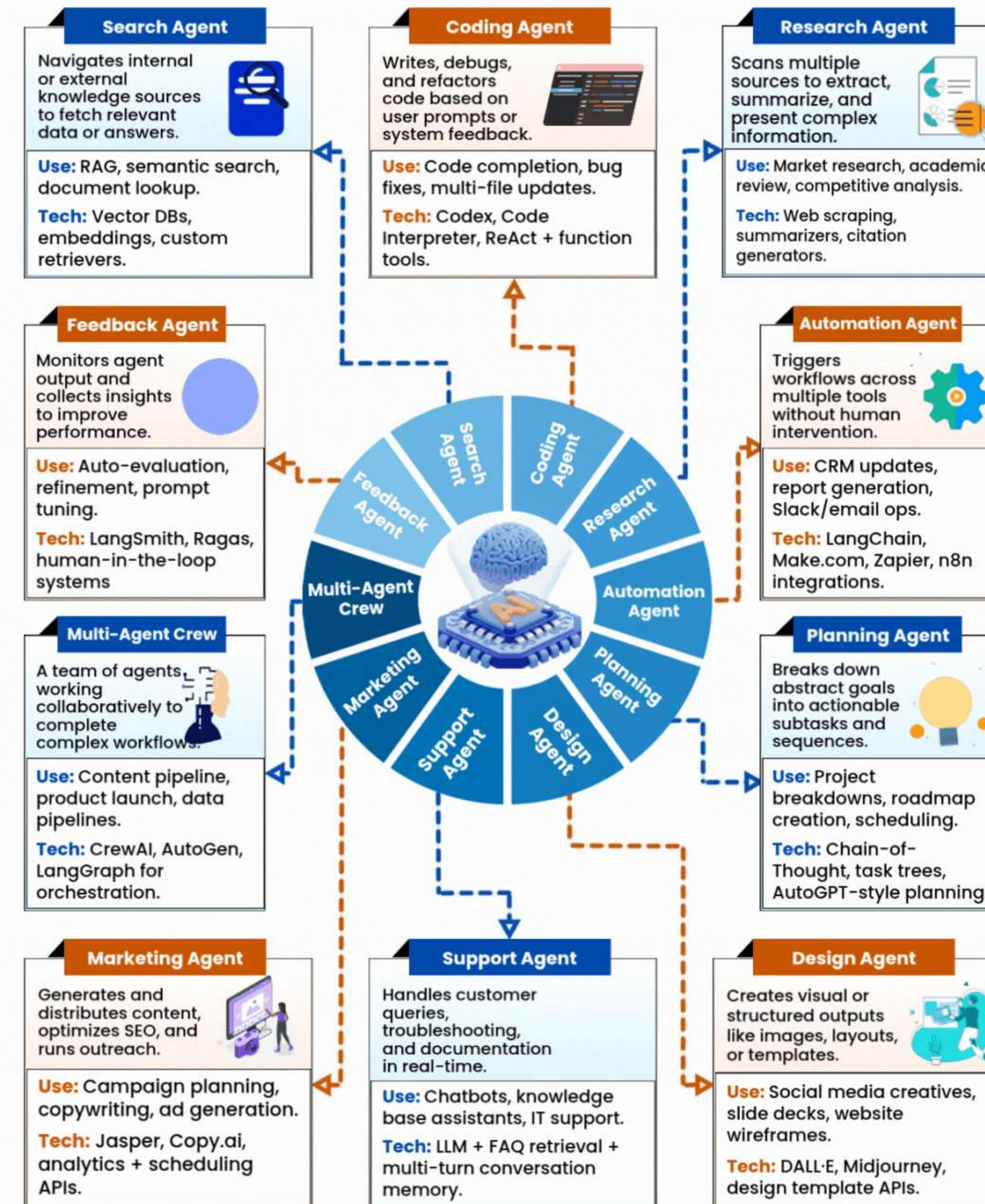
一个 Agent Loop (代理循环)：

User Input ->

1. 向模型发起一个包含其可调用工具的请求
2. 接收来自模型的工具调用
3. 在应用程序端执行代码，使用工具调用的输入
4. 使用工具输出向模型发起第二次请求
5. 接收来自模型的最终响应 (或更多的工具调用)

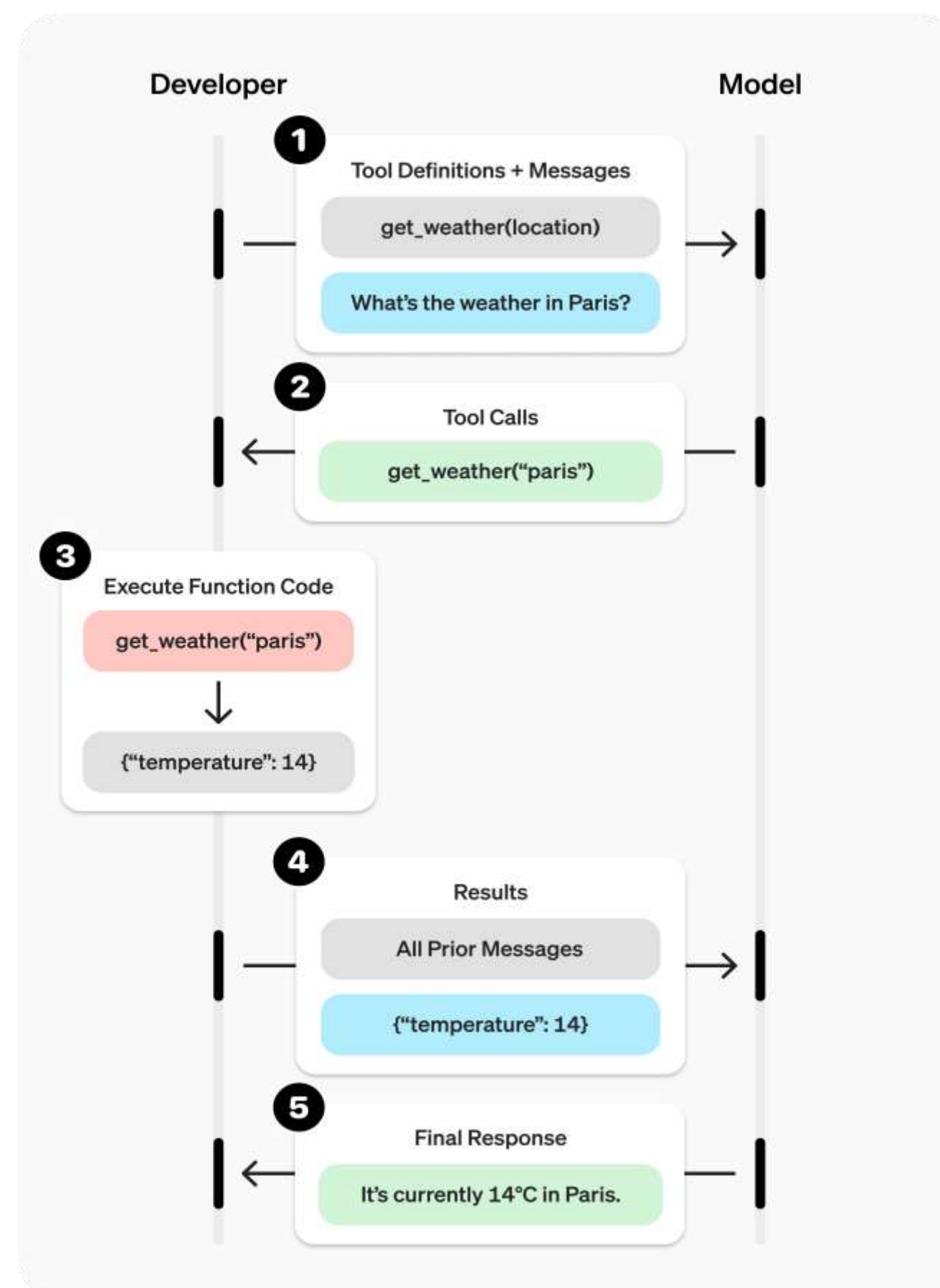
Types of Agent Architectures By Function

Greg Coquillo
Product Leader at AWS



Function calling 技术实现

<https://platform.openai.com/docs/guides/function-calling?api-mode=responses>



```

4
5 tools = [{
6     "type": "function",
7     "name": "get_weather",
8     "description": "Get current temperature for a given location.",
9     "parameters": {
10         "type": "object",
11         "properties": {
12             "location": {
13                 "type": "string",
14                 "description": "City and country e.g. Bogotá, Colombia"
15             }
16         },
17         "required": [
18             "location"
19         ],
20         "additionalProperties": False
21     }
22 }]
23

```


定义函数的最佳实践

- 1 编写清晰详细的功能名称、参数描述和说明。
- 2 应用软件工程最佳实践：使用枚举。
- 3 不要让模型填充你已经知道的参数。
- 4 保持较少的函数数量以提高精度。
- 5 利用模型资源。

FAQ

哪些模型支持函数调用？

函数调用是在2023年6月13日发布gpt-4-turbo时引入，在此日期之前发布的旧版模型不支持函数调用。

并行函数调用支持在2023年11月6日或之后发布的模型上，包括：gpt-4o、gpt-4、gpt-3.5-turbo。

模型可以自己执行函数吗？

不可以。

如何确保模型调用正确的函数？

为函数和参数使用直观的名称和详细的描述。

数据库查询模拟（SQL 查询）

场景：用户问 “销量最高的产品是什么？”

- 1 怎样设计 Function call?
- 2 分成几个步骤?
- 3 怎样定义工具?
- 4 你能描述运行过程吗?

```
if "天气" in user_input and "上海" in user_input:  
    sql = "SELECT temp FROM weather WHERE city='上海' AND date='2024-03-20'"  
    result = db.query(sql)  
    return f"上海昨天的温度是{result}度"
```

数据库查询模拟（SQL 查询）

```
1  tools = [  
2      {  
3          "type": "function",  
4          "function": {  
5              "name": "query_sales_db",  
6              "description": "查询销售数据库",  
7              "parameters": {  
8                  "type": "object",  
9                  "properties": {  
10                     "query_type": {  
11                         "type": "string",  
12                         "enum": ["top_product", "total_revenue", "sales_by_region"],  
13                         "description": "查询类型"  
14                     },  
15                     "region": {"type": "string", "description": "地区 (可选)"}  
16                 },  
17                 "required": ["query_type"]  
18             }  
19         }  
20     ]  
21 ]
```

```
1  "tool_calls": [{  
2      "function": {  
3          "name": "query_sales_db",  
4          "arguments": {"query_type": "top_product"}  
5      }  
6  }]
```

```
1 ▼ def query_sales_db(query_type, region=None):  
2 ▼     if query_type == "top_product":  
3         return {"product": "iPhone 15", "units_sold": 100000}
```


Function calling 的工程意义

维度	价值
解耦	模型只负责“要不要查”，不负责“怎么查”
智能调度	实现多工具协同、条件判断、链式调用
易于扩展	新增工具无需改主逻辑，只需注册
可观察性	能追踪每一步决策（reasoning）和动作（action）
安全性	可在执行前审查 tool_call 参数（防注入）
支持复杂流程	如：先查库存 → 再算价格 → 最后下单

工程化进阶方向（略）

1. 工具注册中心：统一管理所有 tools，支持动态加载
2. 参数校验中间件：防止恶意或错误参数传入
3. 超时与重试机制：工具调用失败自动重试
4. 缓存机制：相同参数避免重复调用
5. 权限控制：某些工具仅限特定角色调用
6. 日志追踪：记录完整的 Agent Loop 流程
7. 可视化调试面板：查看模型决策链路

MCP（Model Context Protocol） 和 Function Calling

核心能力

模型上下文协议（MCP） 是一种开放协议，它标准化了应用程序如何向 LLM 提供工具和上下文。

核心定位差异

Tool Calling：你告诉模型 “你能用哪些工具”

MCP：模型自动发现并使用互联网上的 “标准化工具”

维度	Function / Tool Calling	MCP（Model Context Protocol）
控制权归属	开发者完全掌控工具定义与执行	模型可调用第三方远程服务（非你部署）
工具部署位置	工具在你的代码中（本地或内网）	工具在远程服务器上（如 mcp.stripe.com）
信任模型	完全可信（你自己写的）	需审慎信任第三方（可能恶意）
标准化协议	OpenAI 私有实现（非开放标准）	开放协议（跨平台、多厂商支持）
典型用途	调用内部 API、数据库、计算函数	集成 SaaS 服务（Stripe、Shopify、Twilio 等）

workflow 差异

第 1 步：从 MCP 服务器获取工具列表

第 2 步：调用工具

- （可选的）批准

```
from openai import OpenAI

client = OpenAI()

resp = client.responses.create(
    model="gpt-4.1",
    tools=[
        {
            "type": "mcp",
            "server_label": "deepwiki",
            "server_url": "https://mcp.deepwiki.com/mcp",
            "require_approval": "never",
        },
    ],
    input="What transport protocols are supported in the 2025-03-26 version of the MCP"
)

print(resp.output_text)
```

参考

<https://platform.openai.com/docs/guides/tools-remote-mcp#how-it-works>

场景：

用户说：“帮我生成一个 20 元的支付链接”

Function Calling（传统做法）

MCP（标准协议 + 开放生态）

```
1 tools = [  
2   {  
3     "type": "mcp",  
4     "server_label": "stripe",  
5     "server_url": "https://mcp.stripe.com",  
6     "headers": {  
7       "Authorization": "Bearer sk_live_xxx" # 你的密钥  
8     }  
9   }  
10 ]
```

	Tool Calling	MCP
生态规模	封闭（仅你定义的工具）	开放生态（Cloudflare、Stripe、Shopify、Twilio 等已支持）
可发现性	无公共目录	未来会有 MCP Registry（类似 npm for AI tools）
可组合性	弱（需手动拼接）	强（不同 MCP 服务可协同工作）

MCP vs Tool Calling 何时用？

场景	推荐方案	理由
调用内部数据库、私有 API	Tool Calling	安全、可控、低延迟
快速接入 Stripe/PayPal/Twilio	MCP	无需开发，开箱即用
多个 SaaS 系统联动	MCP	统一协议，避免重复对接
对安全性要求极高（金融、医疗）	MCP + Approval	必须审查每个调用
希望构建通用 AI 工具平台	MCP	支持插件化、可发现、可组合

借助本地部署提高安全性

维度	HuggingFace Transformers	vLLM / SGLang	Ollama
定位	研发/微调	生产级高并发推理	本地快速体验
性能	原生慢	高吞吐（PagedAttention）	快（GGUF量化）
部署	需封装API	内置OpenAI兼容API	开箱即用CLI/API
批处理	不支持	连续批处理	不支持
硬件适配	通用	GPU集群优化	Apple Silicon友好
扩展性	高（灵活）	高（服务化）	低（个人使用）
适用场景	实验开发	企业服务部署	本地运行与测试

Tool Calling 之外的工程强化手段

1. 流式输出 (Streaming)
2. 异步调用与批处理 (Async & Batching)
3. 超时控制与重试策略
4. 上下文管理与长度优化
5. 工具调用审批机制 (Approval)
6. 结构化输出 (JSON Schema)
7. 认证与安全头管理 (Authentication)
8. 限流、熔断与降级策略
9. 监控、日志与链路追踪
10. 输入净化与提示词注入防护

小结

工程反思：

1. 这个技术解决了什么问题？
2. 它的局限是什么？
3. 企业在用吗？怎么用的？

03 LangChain 核心组件详解

✖ 工具层 学会使用工业级框架搭建流程

工程化组件的价值

组件化思想：

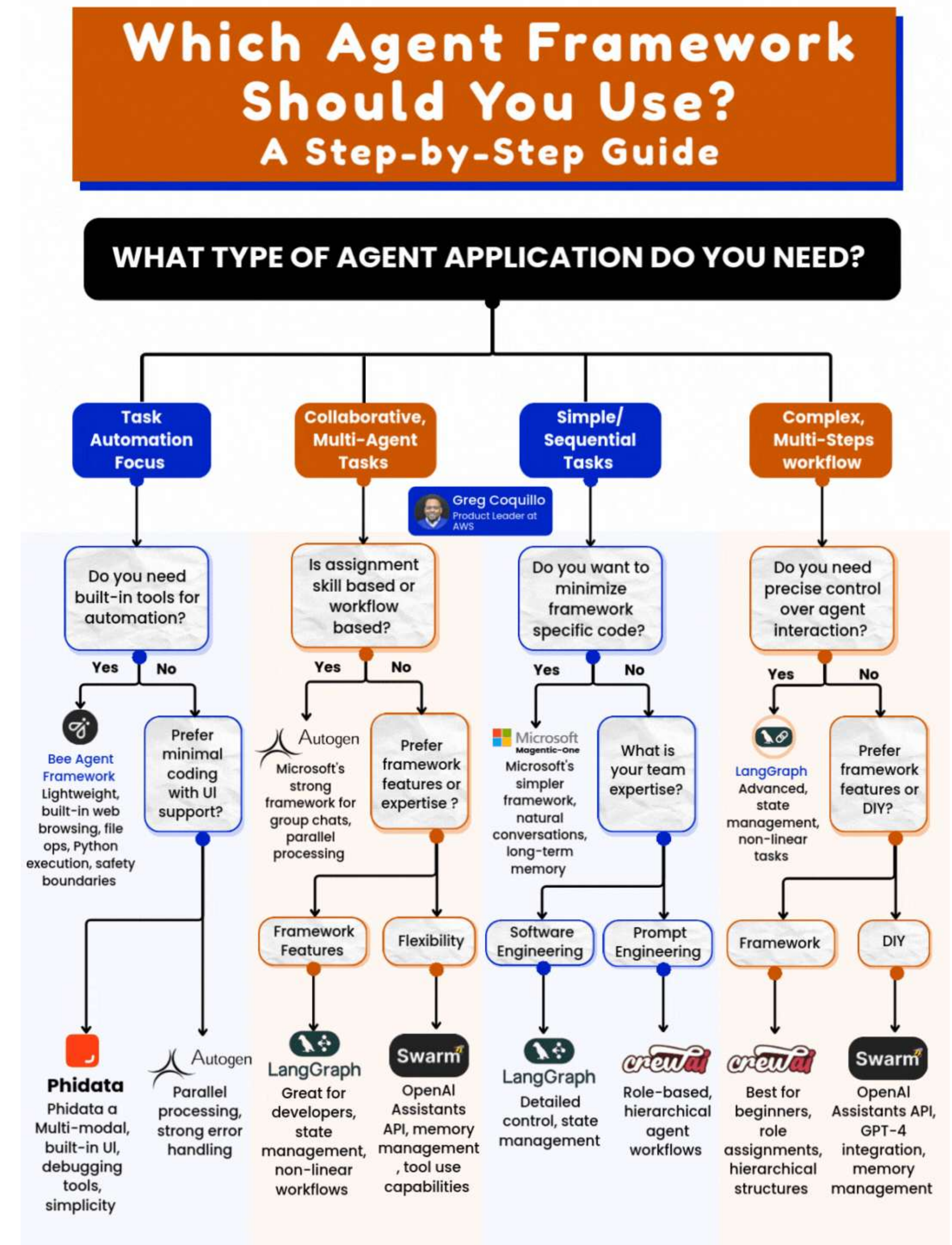
以 LangChain 和 LangGraph 为例

核心区别一句话：

LangChain 是“工具库”，LangGraph 是“流程引擎”。

你用 LangChain 来加载文档、调用模型、执行工具。

你用 LangGraph 来定义：谁先执行？什么条件下重试？失败后走哪个分支？



LangChain组件

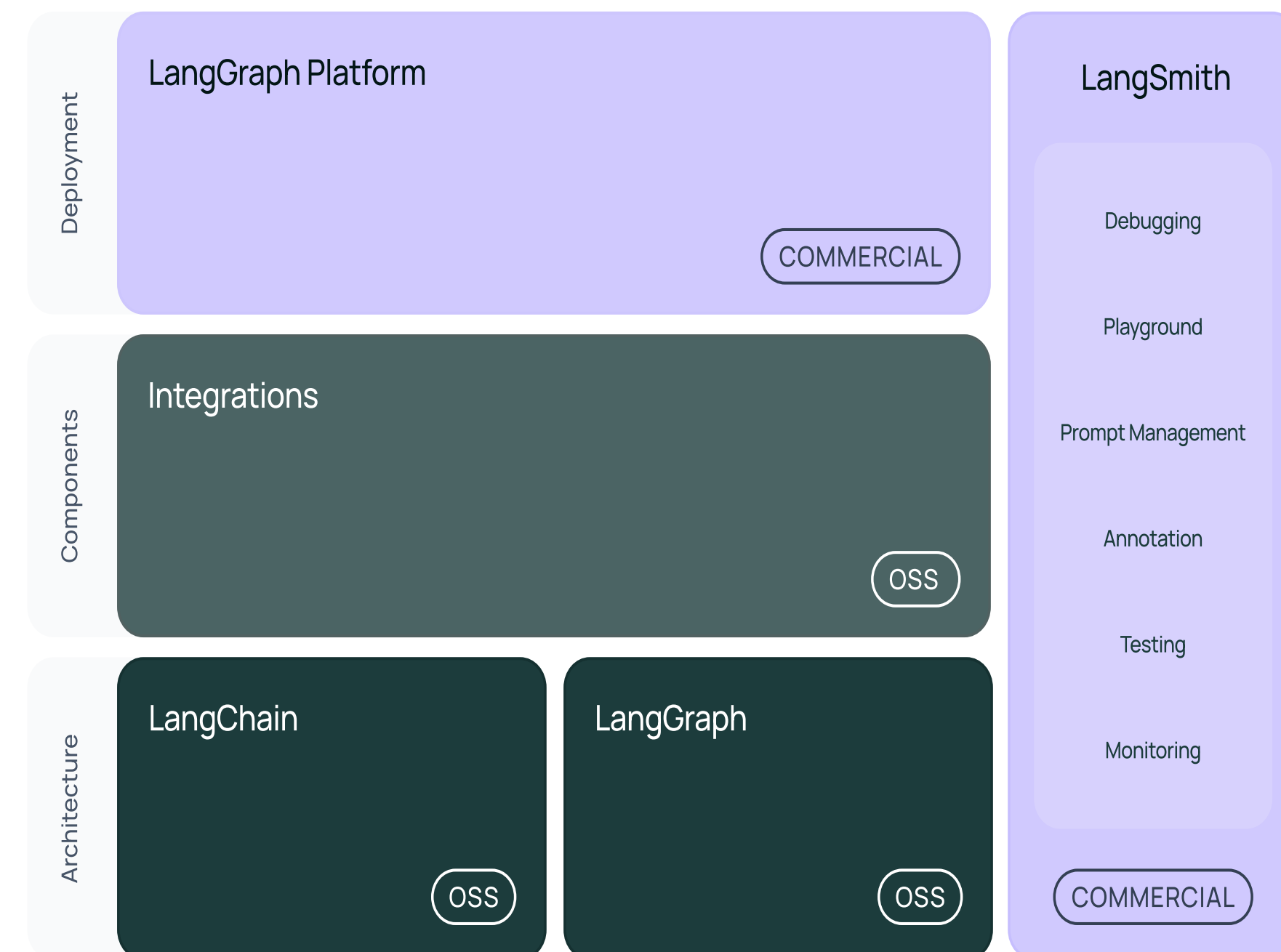
LangChain是一个用于开发由大型语言模型（LLMs）驱动的应用程序的框架。

LangChain简化了LLM应用程序生命周期的每个阶段：

开发：使用**LangChain**的开源组件和第三方集成来构建您的应用程序。使用**LangGraph**构建具有一流流式传输和人工干预支持的持久性代理。

生产化：使用**LangSmith**检查、监控和评估您的应用程序，以便您可以持续优化并自信地部署。

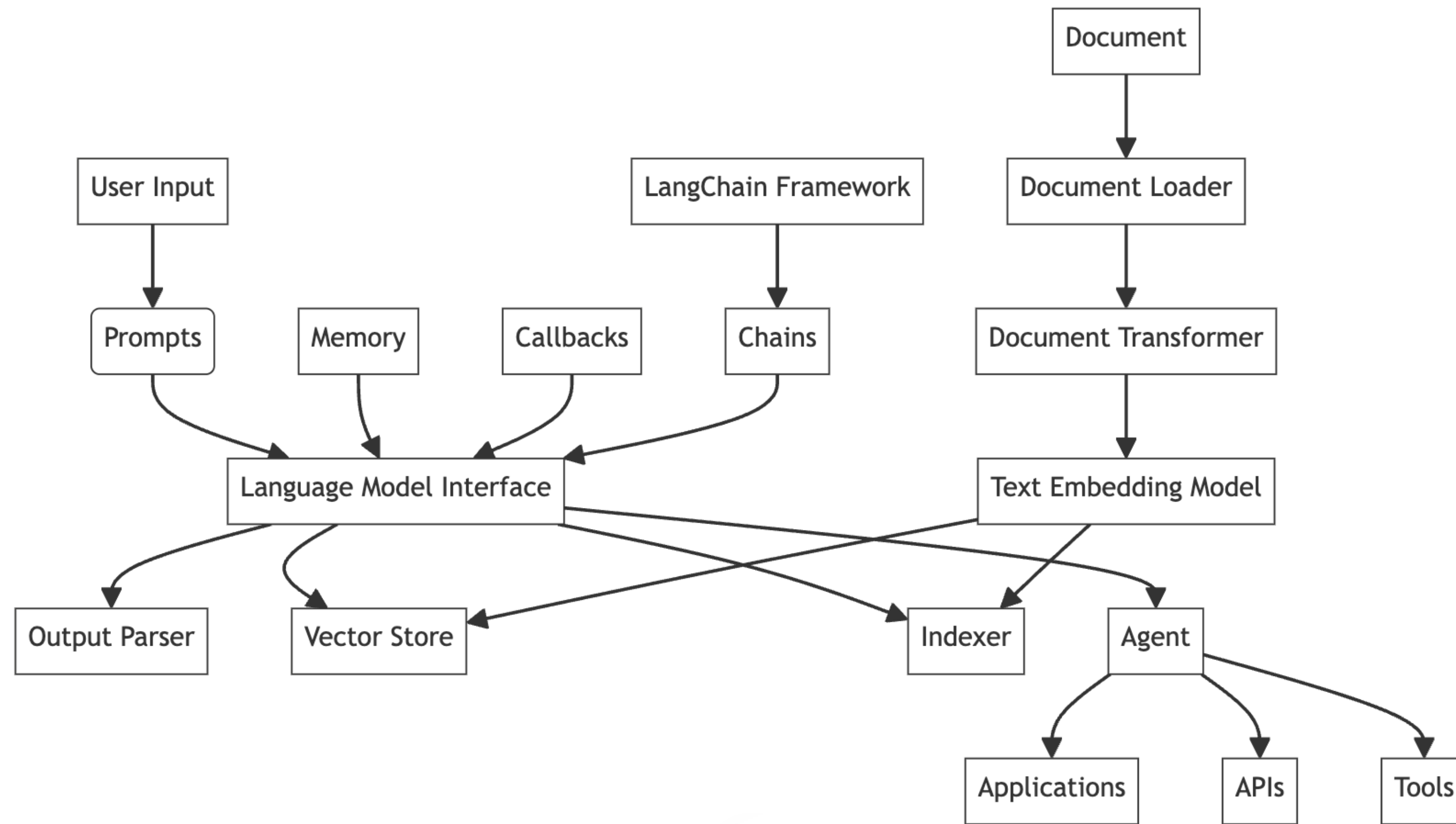
部署：将您的**LangGraph**应用程序转换为生产就绪的API和助手，使用**LangGraph**平台。



LangChain 核心概念和六大模块

- | | |
|------------------------------------|--------------|
| 1. LLM (Large Language Model) | 1. 模型 I/O 模块 |
| 2. PromptTemplate | 2. RAG 模块 |
| 3. Chain (链) | 3. 存储模块 |
| 4. Agent (代理) | 4. 工具模块 |
| 5. Memory | 5. 回调 |
| 6. Document Loader & Text Splitter | 6. LCEL 语法 |
| 7. Embedding Model | |
| 8. Vector Store | |

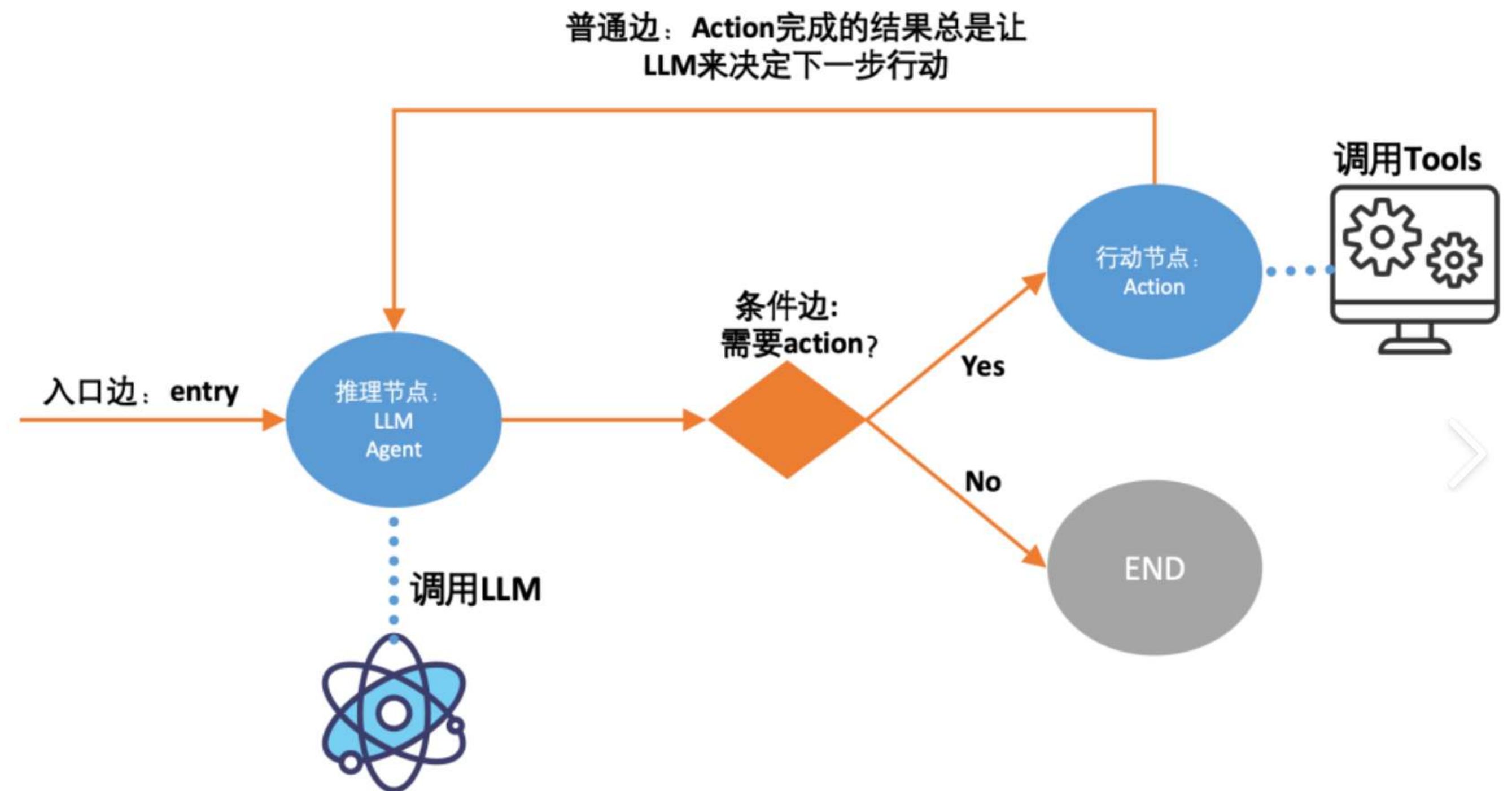
LangChain 六大模块



LangGraph 库

三个关键组件来定义 Agent 行为:

1. 节点 (Node)
2. 边 (Edge)
3. 状态属性 (State Attribute)



小结

LangChain 最大的优势是“乐高式开发”，降低工程复杂度

LangGraph 解决了流程编排。

但企业最有价值的是私有数据，如何让AI“学习”这些内部知识呢？

04 LlamaIndex 与知识增强系统

 数据层 解决“如何让AI知道企业内部信息”问题

RAG 的价值

RAG的核心价值

将外部知识显式注入生成过程，突破模型参数化记忆的静态性与有限性，实现可追溯、可更新、事实增强的推理。

构建解耦的知识架构

知识检索独立于模型，支持实时更新、多源融合与审计追踪，显著降低知识迭代成本，提升系统可靠性与可控性。

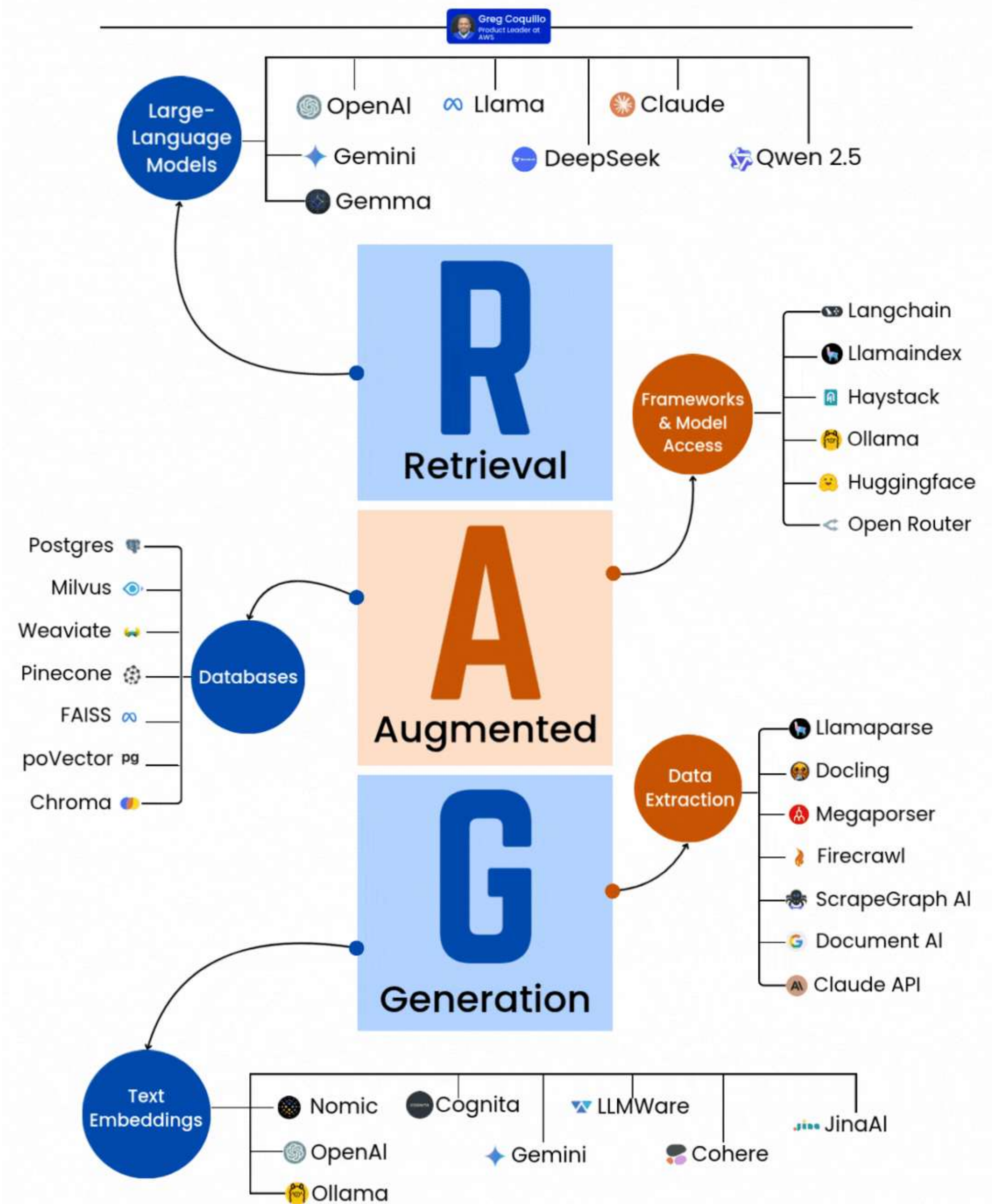
与Agent tool calling 的本质区别

RAG：检索是上下文供给，为生成提供增强依据，输出仍由模型主导，本质是知识增强的推理；

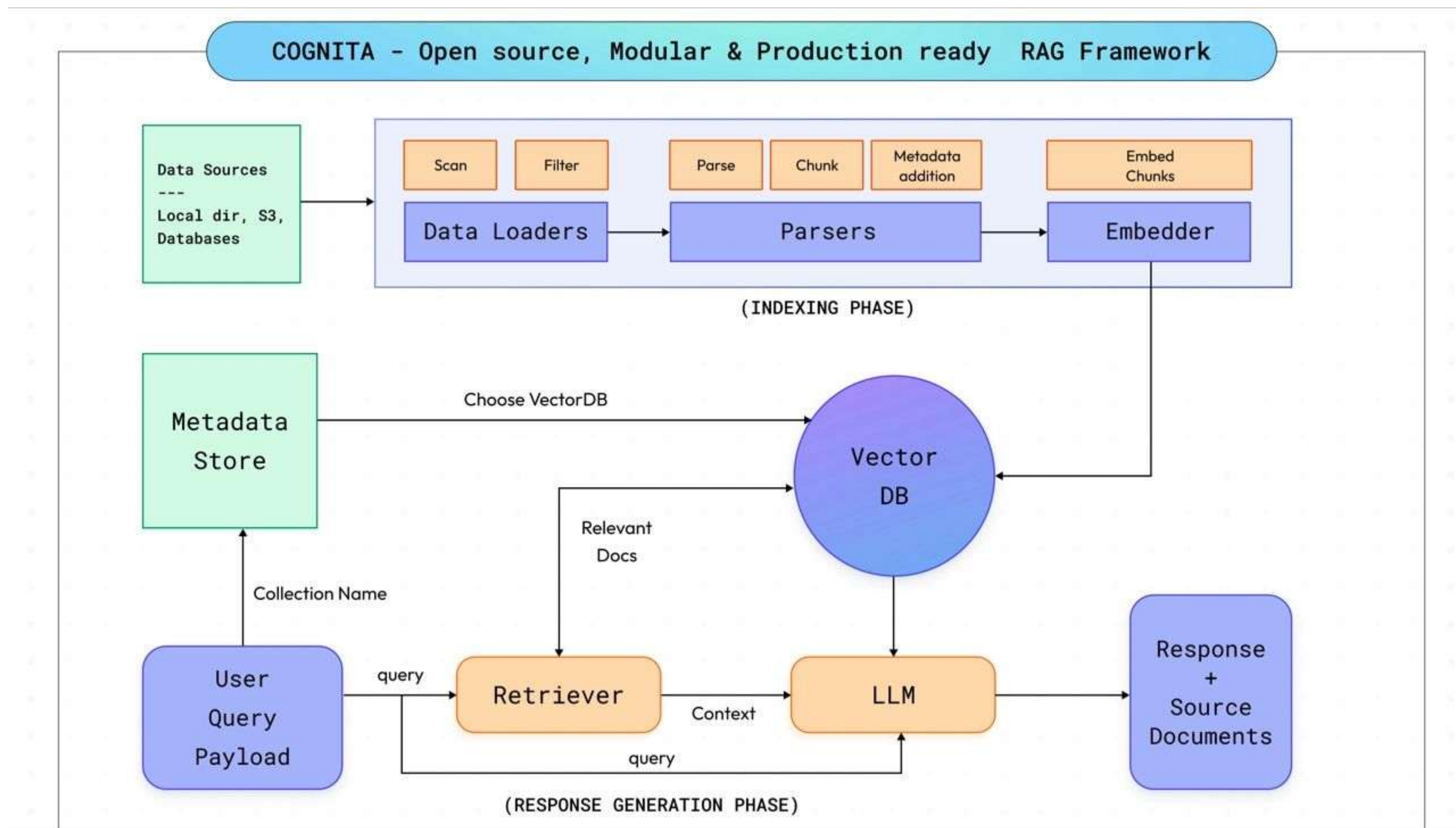
Agent调用工具：动作是任务执行，调用外部API或函数以完成状态变更或获取确定性结果，本质是决策驱动的行动。

RAG重在“知道更多再回答”，Agent重在“做点什么来解决”。

RAG STACK BREAKDOWN FROM RETRIEVAL TO GENERATION

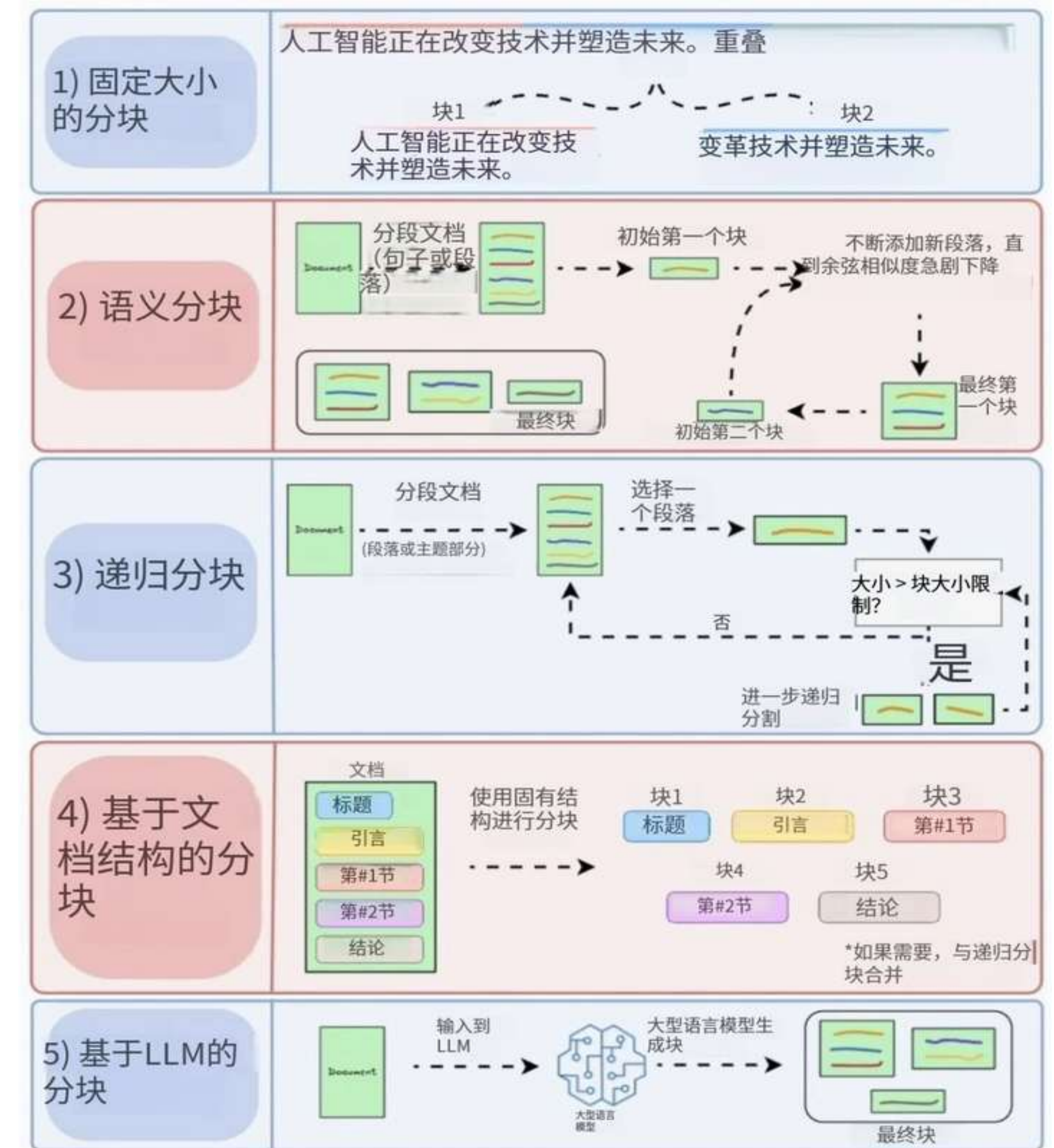


Llama-Index 与 RAG 架构



RAG 能力的增强

- 核心能力：
 - 文档加载器，多格式加载（PDF、网页、SQL、Notion）
 - 向量化索引（embedding + vector store）
 - 查询优化（retrieval-augmented generation）
- 检索 + 生成一体化



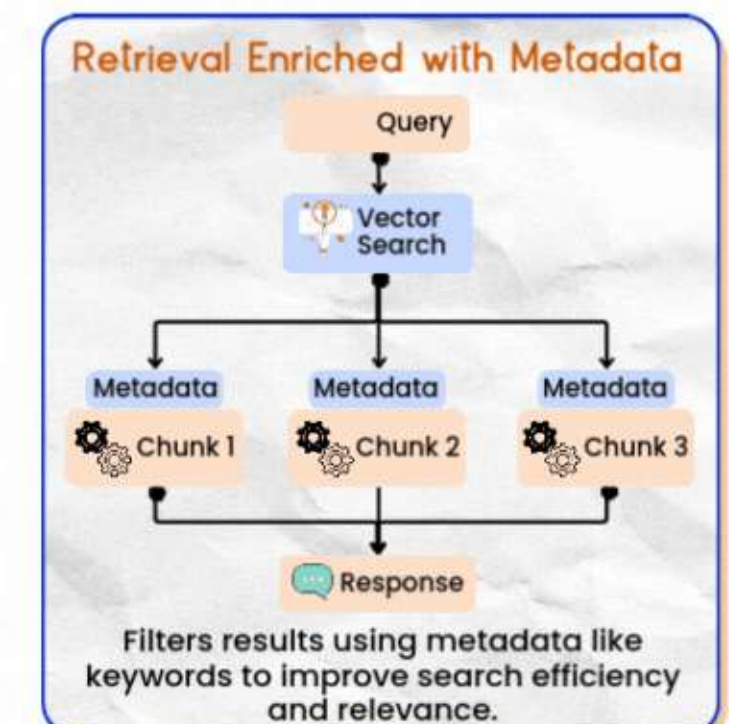
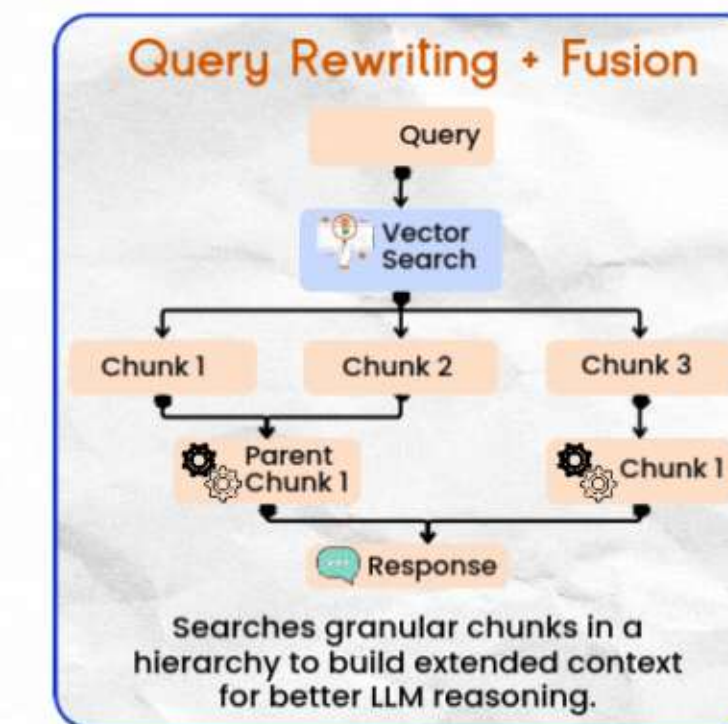
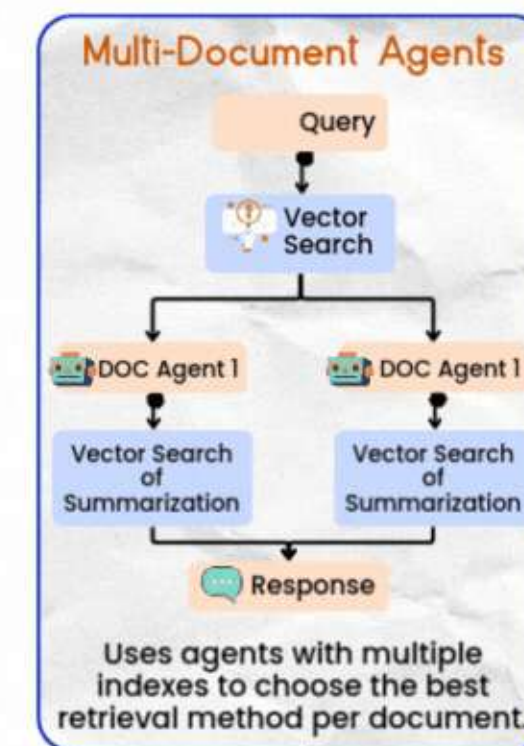
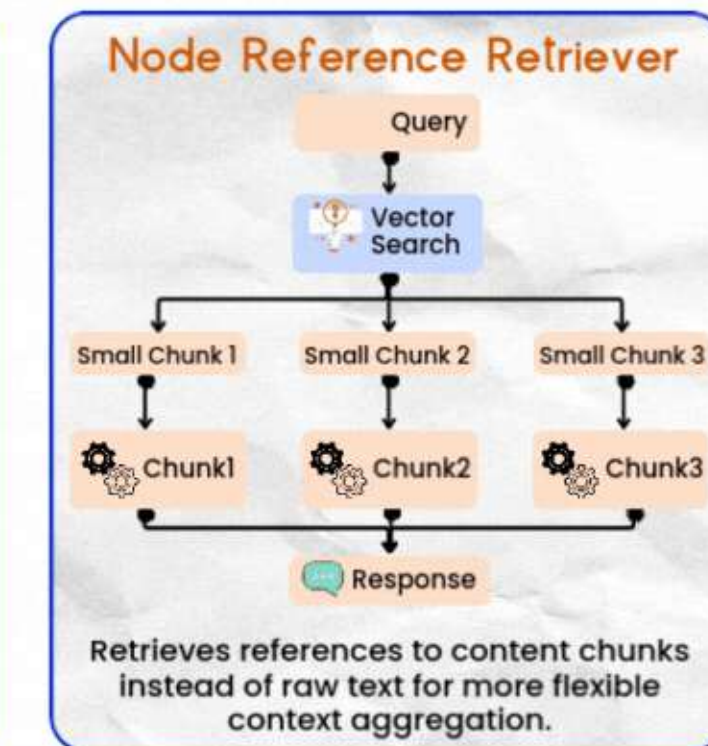
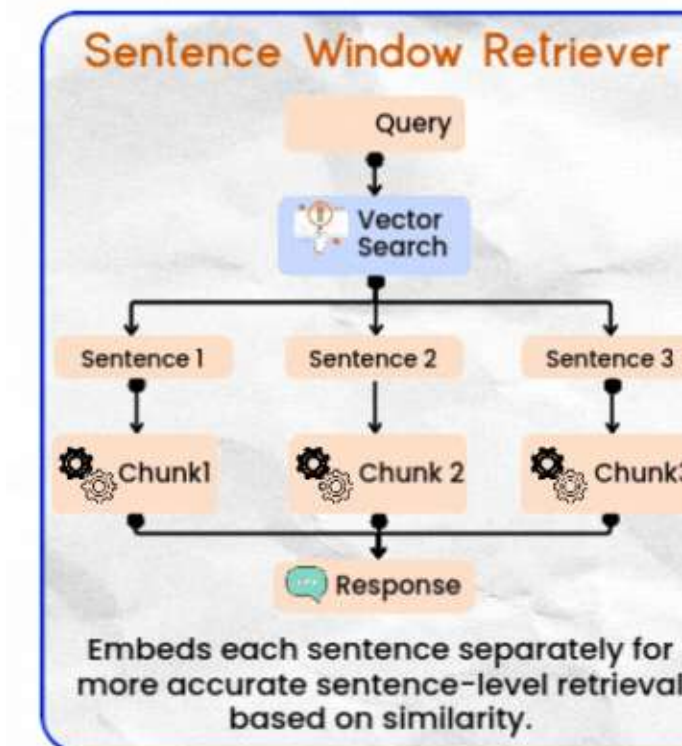
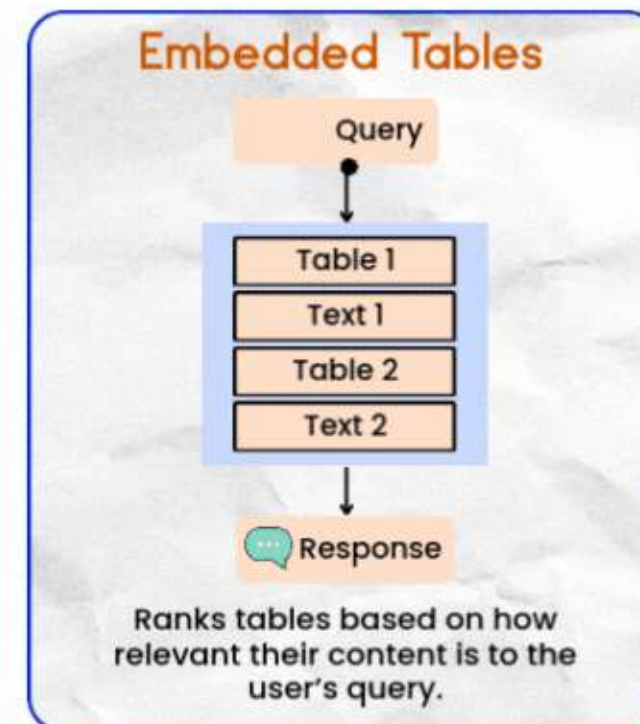
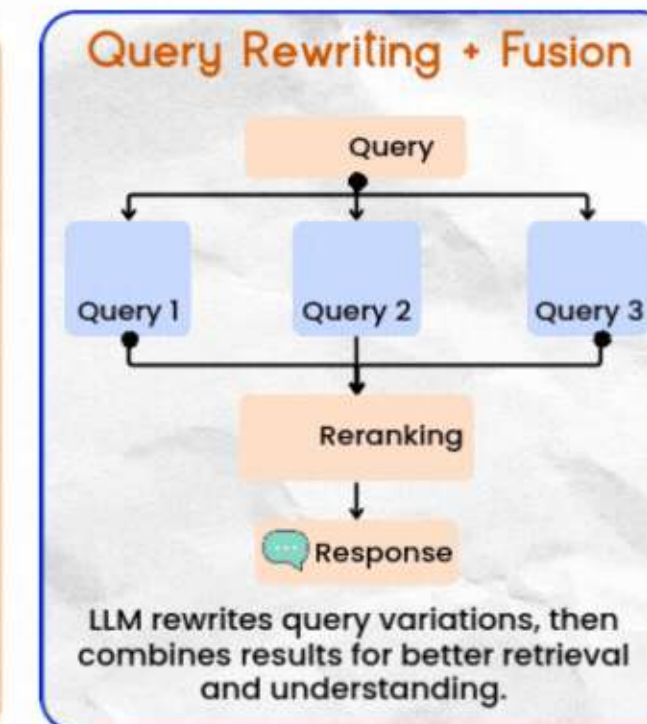
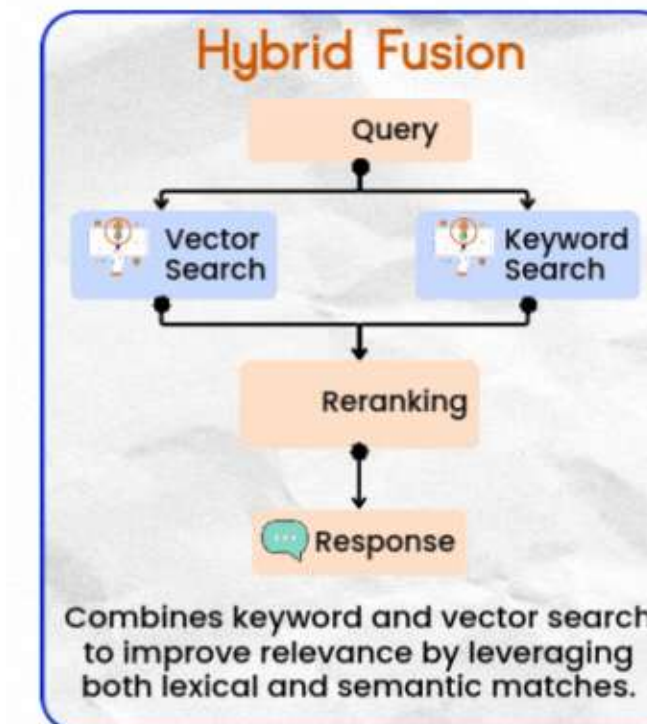
Llama-Index 核心能力

扩展能力:

- 自定义索引结构
 - 通过继承 BaseIndex 实现特定业务场景下的索引结构（如法律条文索引、医学术语图谱），扩展 Index 类型以适应特定业务需求
- 支持混合检索（关键词 + 向量）
- 高级检索策略支持：
 - Hybrid Retrieval: 融合关键词与向量检索，提升准确率
 - Query Transformation: 查询重写、同义词扩展，提升召回率
 - Sub-Question Query Engine: 将复杂问题分解为子问题检索
 - Semantic Chunking: 基于语义的智能分块，避免信息割裂

Advanced Retrieval Techniques

Greg Coquillo
Product Leader at AWS



RAG 能力的“变形”

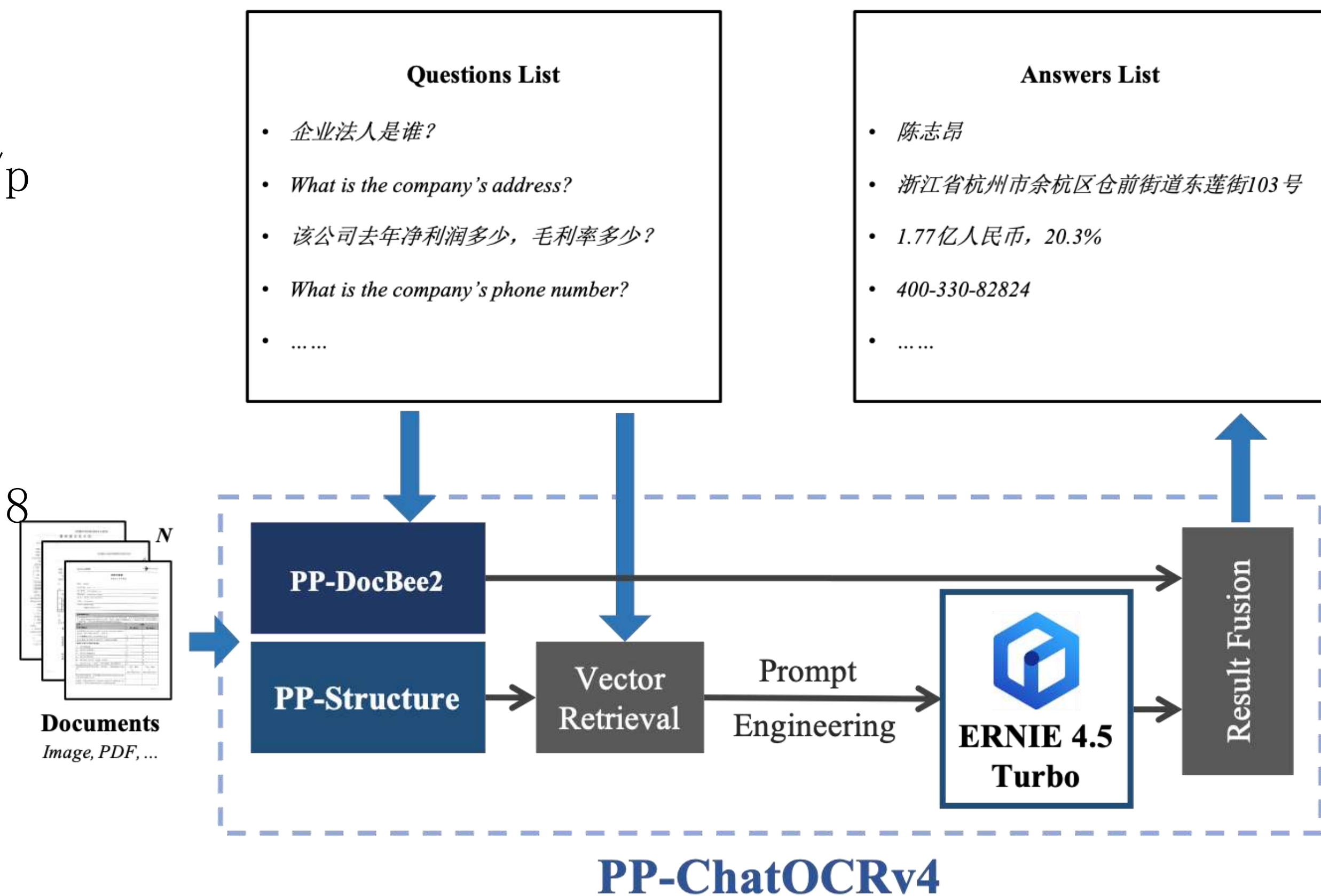
PP-ChatOCRv4

参考地址:

https://www.paddleocr.ai/latest/version3.x/pipeline_usage/PP-ChatOCRv4.html

演示地址:

<https://aistudio.baidu.com/community/app/518493/webUI>



11种新型的 RAG

- 1. InstructRAG
- 2. MADAM-RAG
- 3. CoRAG (Collaborative RAG)
- 4. HM-RAG
- 5. ReaRAG
- 6. HeteRAG
- 7. MCTS-RAG
- 8. CDF-RAG
- 9. Typed-RAG
- 10. NodeRAG
- 11. Hyper-RAG

Types	Purpose	Key Features	Use Case
 InstructRAG	Enhances task planning by integrating RAG with instruction graphs.	Utilizes a multi-agent meta-reinforcement learning framework comprising an instruction graph, a reinforcement learning agent to expand task coverage, and a meta-learning agent for better generalization.	Improves performance in complex task planning scenarios by enabling LLMs to learn from structured instruction sequences
 MADAM-RAG	Handles conflicting evidence in retrieved documents.	Implements a multi-agent system where models debate over multiple rounds, and an aggregator filters out noise and misinformation.	Enhances answer reliability in environments with ambiguous or conflicting information.
 CoRAG (Collaborative RAG)	Facilitates collaborative learning in RAG models across multiple clients.	Introduces a shared passage store where clients jointly train a model, allowing for improved performance in low-resource settings.	Ideal for scenarios requiring collaborative knowledge sharing and model training across different organizations or departments.
 HM-RAG	Facilitates multimodal retrieval and generation through a hierarchical multi-agent framework.	Comprises agents for query decomposition, modality-specific retrieval (text, graphs, web), and answer synthesis, enabling comprehensive query understanding.	Effective in scenarios requiring integration of diverse data types for complex queries.
 ReaRAG	Enhances factual accuracy in reasoning tasks.	Employs a knowledge-guided reasoning approach that iteratively constructs reasoning chains, reducing unnecessary iterations and improving robustness.	Suitable for complex question-answering tasks where factual correctness is paramount.
 HeteRAG	Enhances retrieval and generation by decoupling knowledge chunk representations.	Utilizes multi-granular views for retrieval and concise chunks for generation, along with adaptive prompt tuning to improve efficiency and effectiveness.	Beneficial for applications needing precise and efficient information retrieval and generation.
 MCTS-RAG	Improves reasoning capabilities of small language models in knowledge-intensive tasks.	Integrates Monte Carlo Tree Search (MCTS) with RAG to refine reasoning paths through an iterative decision-making process.	Effective in scenarios where small models need to perform complex reasoning with limited computational resources.
 CDF-RAG	Enhances causal reasoning in RAG systems.	Incorporates causal graphs and dynamic feedback mechanisms to refine queries and validate responses against causal pathways.	Suitable for applications requiring understanding of cause-and-effect relationships, such as policy analysis or scientific research.
 Typed-RAG	Addresses non-factoid question answering by considering question types.	Classifies questions into types (e.g., debate, experience, comparison) and applies aspect-based decomposition to enhance retrieval and generation.	Improves responses to open-ended questions by tailoring strategies based on question type.
 NodeRAG	Integrates heterogeneous graph structures into RAG workflows.	Introduces well-designed graph structures to seamlessly incorporate graph algorithms, improving performance on multi-hop and open-ended question answering.	Ideal for tasks involving complex relational data and requiring structured reasoning.
 Hyper-RAG	Combats hallucinations in LLM outputs using hypergraph structures.	Employs hypergraphs to capture both pairwise and complex relationships in domain-specific knowledge, enhancing factual accuracy and reducing hallucinations.	Particularly useful in high-stakes fields like medicine, where accuracy is critical.

RAG 丰富的生态

构建RAG 所需的各类工具和技术栈，涵盖嵌入模型、向量数据库、集成框架、LLM插件等。

分工明确：

Embeddings for RAG: 提供高质量的嵌入模型，用于将文本转化为向量，支持高效检索。

Vector Databases: 存储和管理向量数据，支持快速检索和语义搜索。

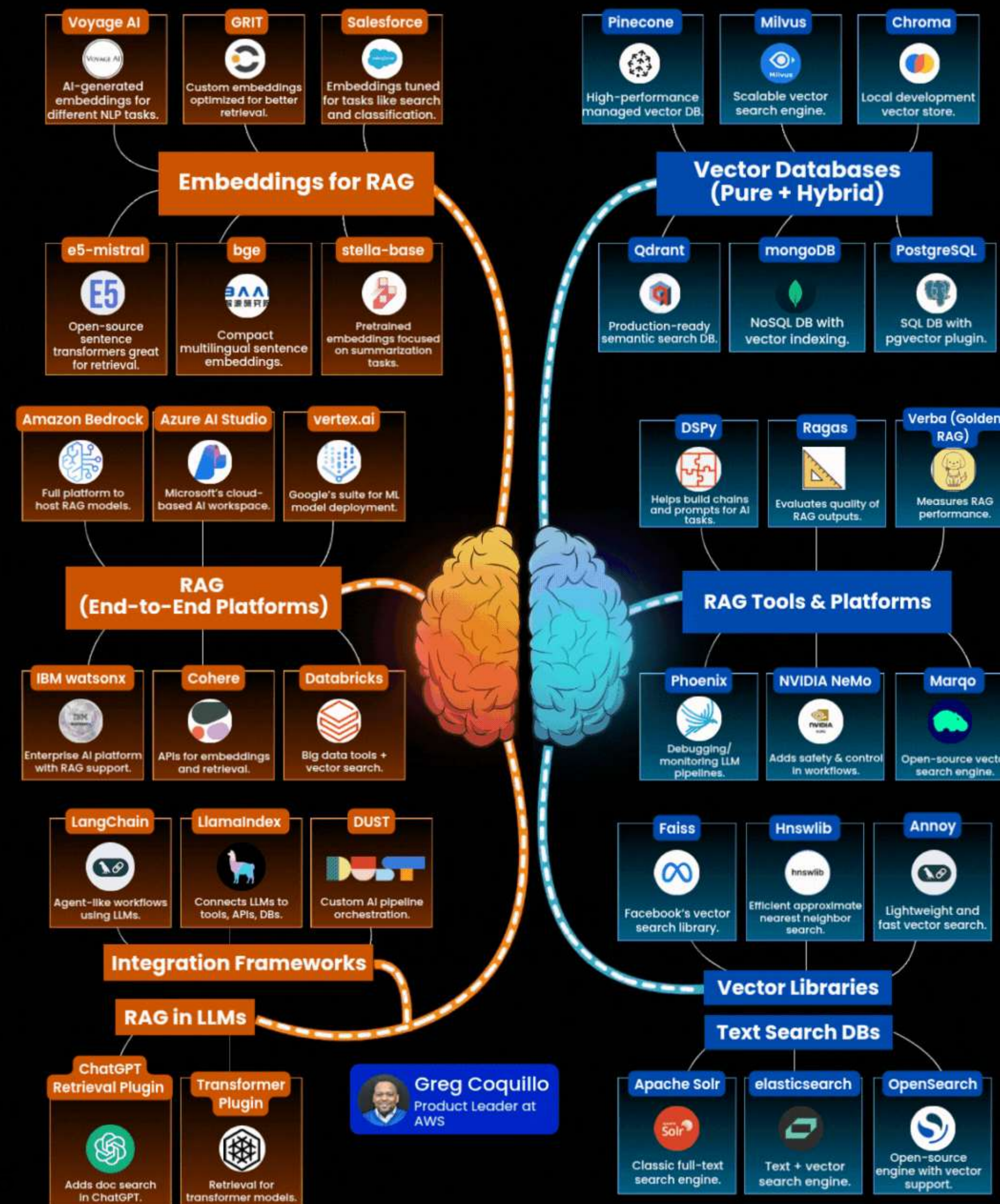
RAG Tools & Platforms: 提供端到端的RAG解决方案，简化开发流程。

Integration Frameworks: 连接不同组件，实现无缝集成。

RAG in LLMs: 将RAG能力嵌入到大语言模型中，增强其知识获取与推理能力。

工具多样，覆盖从研发到部署的全生命周期。生态开放，推动RAG技术持续演进。

TOOLS FOR BUILDING RAG



05 Prompt Engineering 高阶技巧

- 🗣️ 控制层 提升AI推理质量与可控性

Prompt

目的：提升AI推理质量与可控性，精细化控制模型的推理质量、行为边界和输出风格。

比如：设计 Tool Calling Prompt → 统一接口规范，降低维护成本

Prompt 不只是改变大模型的输出内容与格式，也是驱动 Agent 的最重要手段。

比如：

意图识别、

任务分解、

外部工具调用等关键功能。

Prompt

- 推理增强技术：
 - 思维链（Chain-of-Thought）：
 - “Let’s think step by step”
 - 示例：数学题、逻辑推理
- 反思机制（Self-Reflection）：模型自我评估与修正
 - 模型自我评估输出质量
 - 使用 Critic Agent 或 prompt 设计

Prompt

- 提示模板工程:
 - LangChain PromptTemplate / Jinja2

```
1 from langchain_core.prompts import PromptTemplate
2
3 # 使用 Jinja2 语法, 支持 if/else
4 template = """
5 你是一个客服助手, 请根据用户问题提供帮助。
6
7 用户问题: {{ query }}
8
9 {% if product_info %}
10 产品信息: {{ product_info }}
11 请基于以上信息回答。
12 {% else %}
13 该产品暂无详细信息。
14 {% endif %}
15
16 回答:
17 """
18
19 # 创建支持 Jinja2 的 PromptTemplate
20 prompt = PromptTemplate.from_template(template, template_format="jinja2")
21
22 # 格式化输出 (带条件)
23 formatted_prompt = prompt.invoke({
24     "query": "iPhone 15 有货吗?",
25     "product_info": "iPhone 15 256GB 版本有现货, 售价 6999 元。"
26 })
27
28 print(formatted_prompt.text)
```

Prompt

- 提示模板工程：
 - 支持变量注入、条件判断

```
1 template = """  
2 你是一个旅游推荐官。  
3  
4 目的地: {{ city }}  
5 天数: {{ days or 3 }}  
6 预算等级: {{ budget_level | default('中等', true) }}  
7  
8 请推荐一个 {{ days }} 天的行程, 预算 {{ budget_level }}。  
9 """  
10  
11 prompt = PromptTemplate.from_template(template, template_format="jinja2")  
12  
13 # 测试: 只传 city, 其他用默认值  
14 formatted = prompt.invoke({"city": "京都"})  
15 print(formatted.text)
```


Prompt

- 提示模板工程:
 - 动态 Prompt 生成（函数化构造）

```
1 def create_dynamic_prompt(intent: str):
2     templates = {
3         "refund": """
4         你是一名售后客服。
5         用户申请退款，订单号: {{ order_id }}, 原因: {{ reason }}。
6         请礼貌回复并说明处理流程。
7         """,
8         "support": """
9         你是一名技术支持。
10        用户遇到问题: {{ issue }}。
11        环境: {{ environment }}。
12        请提供排查建议。
13        """,
14        "recommend": """
15        你是一名推荐助手。
16        用户偏好: {{ preference }}。
17        请推荐一个 {{ preference }} 风格的产品。
18        """
19    }
20    return PromptTemplate.from_template(templates.get(intent, "请回答: {{ input }}"))
21
22 # 根据用户意图动态选择模板
23 user_intent = "refund"
24 prompt = create_dynamic_prompt(user_intent)
25
26 input_data = {
27     "order_id": "ORD123456",
28     "reason": "商品发错"
29 }
30
31 formatted = prompt.invoke(input_data)
32 print(formatted.text)
```

Prompt

- 提示模板工程:
- 意图识别

```
4 # 步骤 1: 用 LLM 判断用户意图
5 intent_prompt = PromptTemplate.from_template(
6     "判断用户问题的意图, 仅返回一个关键词: refund, support, recommend, general\n"
7     "用户问题: {{ query }}\n"
8     "意图: "
9 )
10
11 llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0)
12
13 # 完整链: 用户输入 → 判断意图 → 动态生成 Prompt → 最终回答
14 def build_smart_agent_chain():
15     # 意图识别链
16     intent_chain = intent_prompt | llm | StrOutputParser()
17
18     def route_prompt(data):
19         query = data["query"]
20         intent = data["intent"]
21
22         # 动态选择并格式化 Prompt
23         dynamic_prompt = create_dynamic_prompt(intent)
24         return dynamic_prompt.invoke({"input": query, **data})
25
26     # 主链: 先识别意图, 再生成回答
27     full_chain = (
28         {"query": lambda x: x["query"]}
29         | intent_chain.assign(intent=lambda x: x) # 保留原始输入
30         | route_prompt
31         | llm
32         | StrOutputParser()
33     )
```


Prompt

- 外部工具调用提示设计 (Tool Calling Prompt) :
 - 明确指令引导模型选择正确工具 (如: “若需查询订单, 请调用 `query_order_tool`”)
 - 提供清晰参数描述与示例, 减少误调用
- Few-shot Prompting 与 Example Selector
- ****安全与管理****:
 - Guardrails / Safe Prompting: 防御提示注入、防止越权
 - Prompt 版本管理与 A/B 测试: 科学迭代提示, 评估效果

Prompt

案例：设计“客服意图识别 + 工具调用”复合 Prompt

你是一个智能客服助手，负责理解用户问题、准确识别其意图，并在必要时调用合适的工具来解决问题。请严格按照以下流程执行：

步骤 1：意图识别

分析用户输入，判断其核心意图。可识别的意图包括但不限于：

- 查询订单状态
- 申请退货/退款
- 咨询产品信息
- 投诉或建议
- 账户问题（登录、密码等）
- 物流信息查询
- 优惠券/促销活动咨询
- 其他（无法归类时使用）

输出格式：{"intent": "意图类别"}

步骤 2：参数提取

从用户输入中提取执行该意图所需的参数。例如：

- 订单号
- 产品名称或 ID
- 用户手机号/邮箱
- 问题描述
- 时间范围等

输出格式：{"parameters": {"参数名": "值", ...}}

步骤 3：工具调用决策

根据意图和参数完整性，决定是否调用工具：

- 若参数完整且需外部系统支持 → 生成工具调用请求
- 若参数缺失 → 请求用户补充信息

- 若无需工具（如简单问答）→ 直接回复

可用工具列表：

1. `query\_order\_status(order\_id: str)` → 查询订单状态
2. `get\_logistics\_info(order\_id: str)` → 获取物流信息
3. `refund\_apply(order\_id: str, reason: str)` → 提交退款申请
4. `search\_product(keyword: str)` → 搜索产品信息
5. `check\_coupons(user\_id: str)` → 查询用户优惠券
6. `submit\_feedback(content: str, contact: str)` → 提交反馈建议

输出格式（工具调用）：

{"tool_call": {"name": "工具名", "arguments": {"参数": "值"}}

输出格式（需补充信息）：

{"ask_for": ["缺少的参数1", "缺少的参数2"]}

输出格式（直接回复）：

{"response": "自然语言回复内容"}

示例交互：

用户：我的订单什么时候发货？单号是 ORD123456789。

AI：

```
{"intent": "查询订单状态"}
{"parameters": {"order_id": "ORD123456789"}}
{"tool_call": {"name": "query_order_status", "arguments": {"order_id": "ORD123456789"}}
```

现在开始处理新请求：

用户：{用户输入}

AI：

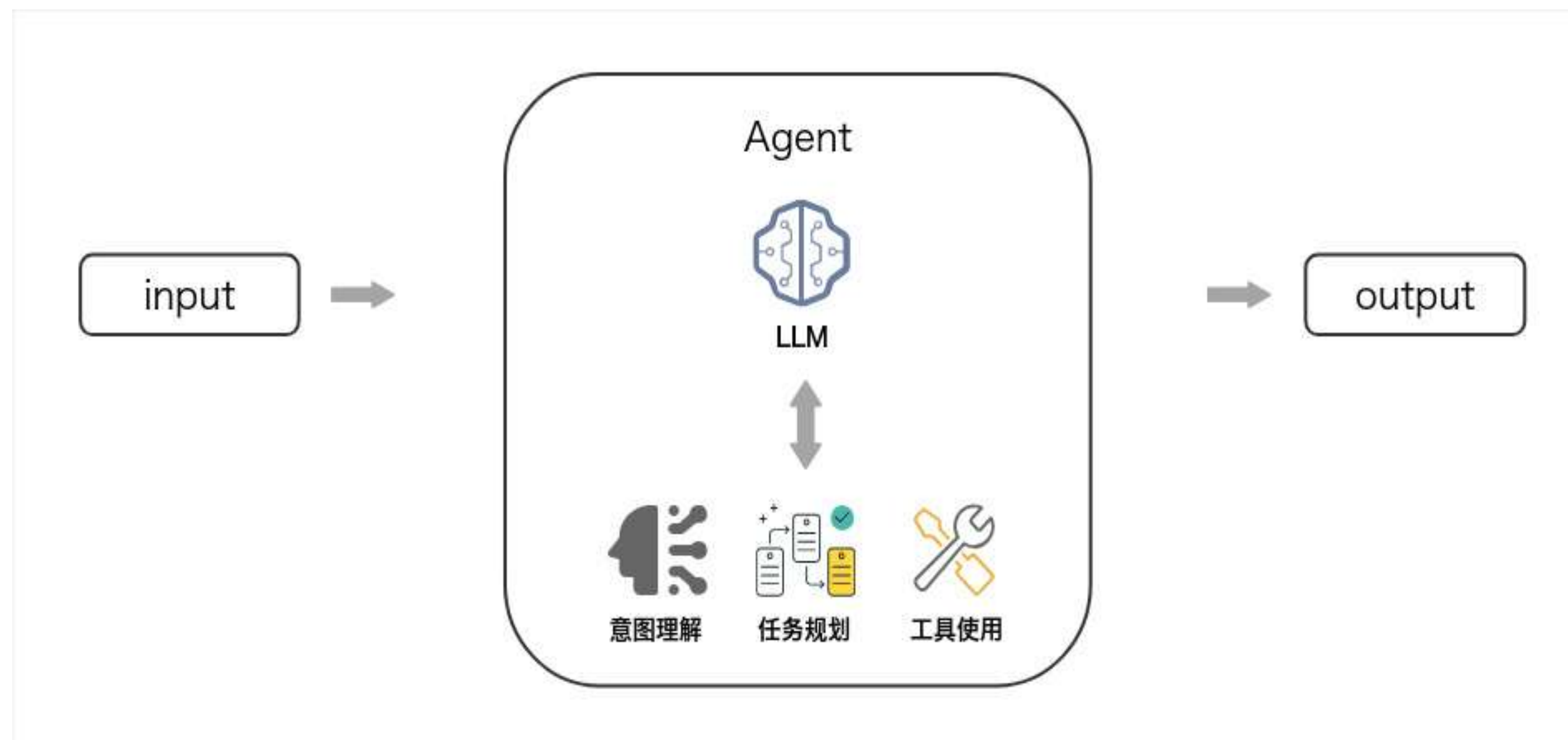
06 Agent 设计与多轮对话逻辑



架构层 构建有状态、可追踪的智能体

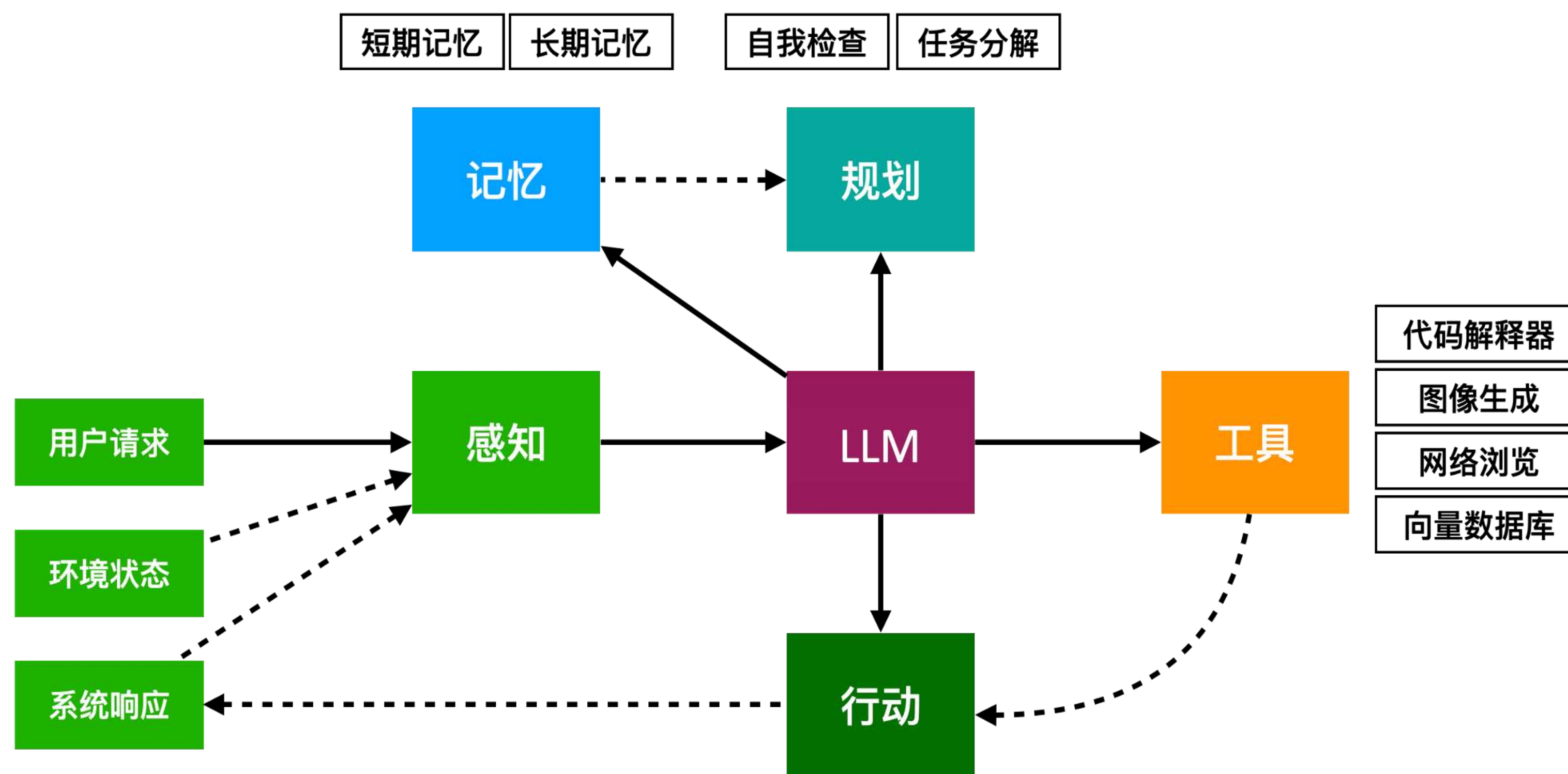
Agent 设计目标

目的：构建有状态、可追踪、可控的智能体，处理复杂的多轮交互任务。



Agent 的本质

感知 + 决策 + 行动 + 记忆



Agent 的本质

一、状态管理：

- 对话状态机 (FSM) $M = (S, I, T, s_0)$

其中 S 为状态集, I 为输入事件, $T \subseteq S \times I \rightarrow S$ 为转移函数, s_0 为初始状态。

- Dialogue State Tracker (DST) 概念引入

二、上下文保持：

- Memory 设计模式
 - Working Memory context
 - Session ID + 用户画像存储
 - Long-term

Agent 的本质

三、对话核心技术：

- Intent Recognition + Slot Filling 槽位：理解用户意图并提取关键信息
- Failure Handling：失败重试、降级策略、错误兜底
- Conversation 生命周期：Session ID管理、上下文清理

四、消息协议设计方法论：

- 标准化字段：role, content, tool_calls, metadata
- 扩展字段建议：session_id, task_id, intent, confidence
- 错误码规范：定义通用错误类型（如 TOOL_NOT_FOUND, PARAM_MISSING）
- 支持异步响应与超时机制

Agent 的本质

示例流程

用户提问 → Agent 分解任务 → 调用工具 → 多轮交互 → 返回结果

设计建议：结合 FSM 实现任务状态流转（如：待确认 → 查询中 → 已完成）

07 Autogen 与多Agent协作机制



协同层 实现团队式AI协作

多 Agent 协作机制

多 Agent 系统（Multi-Agent System, MAS）是一种由多个自主、交互的智能体（Agent）组成的分布式系统，每个Agent具有一定的自主性、感知能力、决策能力和通信能力。

技术特点：

多Agent系统相比单Agent，具备天然的并行性与分布式架构，显著提升系统响应速度与资源利用率。

单点故障不影响整体运行，鲁棒性与容错性远超集中式单Agent系统。

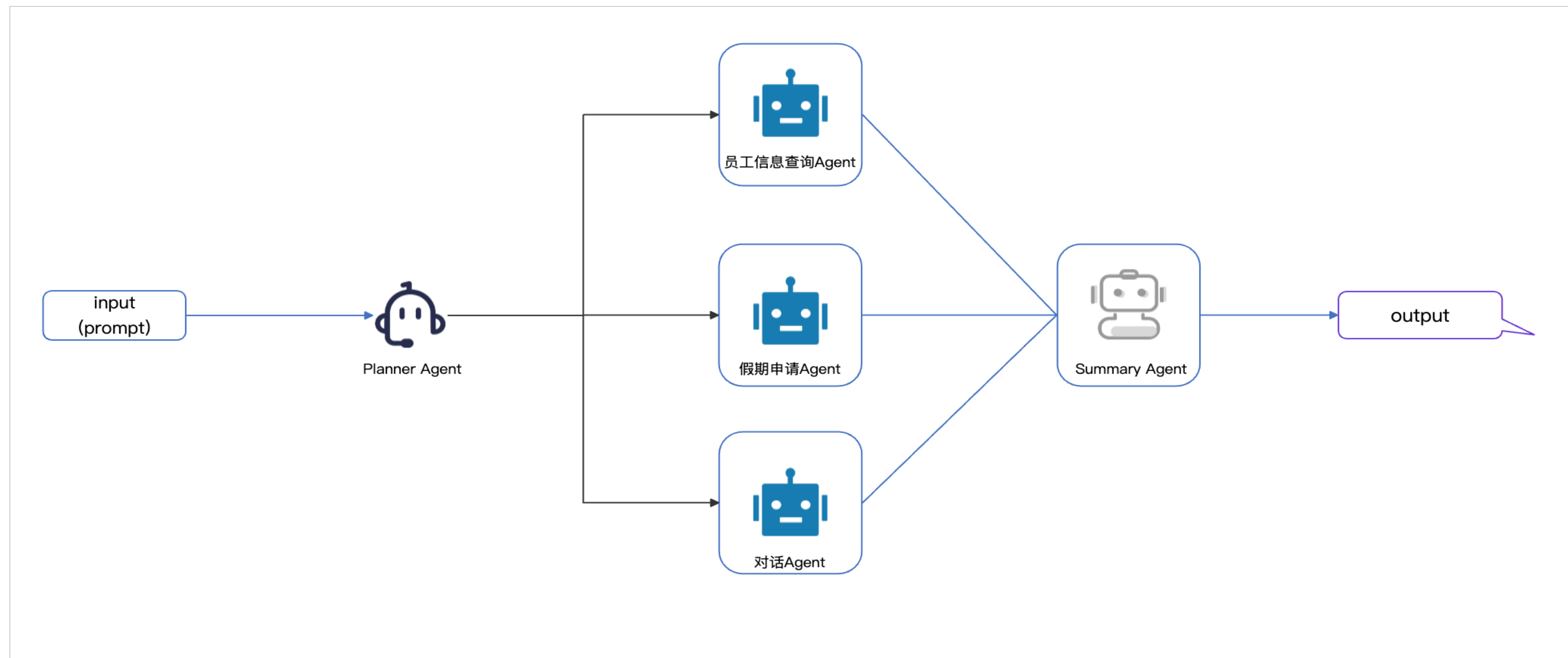
通过协商、竞争与协作机制，可实现全局优化与自适应调整。

模块化设计便于系统扩展与维护，适用于大规模复杂场景。

在智能制造、交通、能源等工程领域，展现出更高的灵活性与可部署性。

Agent 角色定义

- UserProxyAgent（用户代理）
- AssistantAgent（助手）
- Custom Roles（客服、技术员、经理等）



Agent 相关概念

- 协作模式：
 - Group Chat: 多Agent轮流发言，达成共识
 - Debate 机制: Agent间进行辩论以提升决策质量
- 通信机制：
 - Message Passing: 基于 `send()` / `receive()` 的异步消息机制
 - 工具调用传递: Agent A 发起调用，结果由 Agent B 处理或验证
 - 反馈循环 (Feedback Loop): Critic Agent 对输出进行评估并提出改进建议
 - 支持 `human_input_mode` 实现人工干预
- ****高级定制与控制****:
 - Customizable ConversableAgent: 深度自定义Agent行为逻辑
 - Nested Group Chat: 实现“分组讨论”再汇报的复杂协作
 - Human-in-the-loop: 关键节点引入人工审核与决策

Agent 案例

构建一个基于角色分工的多Agent协作系统，模拟真实客服场景中跨环节协同处理用户订单问题：

Agent A（订单专员）：查询用户订单状态（如支付、发货、退款）

Agent B（物流专员）：获取当前物流轨迹与预计送达时间

Agent C（客服主管）：汇总信息，生成自然、准确、一致的用户回复

答：用Autogen拆解订单问题。

08 轻量微调方法比较（LoRA/P-Tuning等）

🔧 个性化层 理解何时及如何定制模型行为

微调的目的

微调（Fine-tuning）的核心目的，是在通用大模型（General-Purpose LLM）的基础上，通过小规模、特定领域的数据训练，使其行为更贴合具体业务场景的需求。

微调不是为了“让模型学会新知识”（那是 RAG 的任务），而是为了改变模型的“表达方式”、“决策偏好”和“输出风格”，实现深度定制化。

目的	说明	典型案例
1. 定制输出格式与风格	让模型固定按照某种结构、语气、术语输出	客服话术标准化、报告生成模板化
2. 学习领域语言（Domain Adaptation）	掌握行业术语、缩写、表达习惯	医疗、法律、金融等专业领域
3. 优化任务表现（Task-Specific Performance）	在特定任务上超越通用模型	分类、摘要、代码生成等
4. 减少提示词工程依赖	降低对复杂 prompt 的依赖，提升鲁棒性	无需写长篇 system prompt 也能正确响应
5. 隐私与数据闭环	敏感数据不出域，训练专属模型	企业内部流程、客户沟通记录

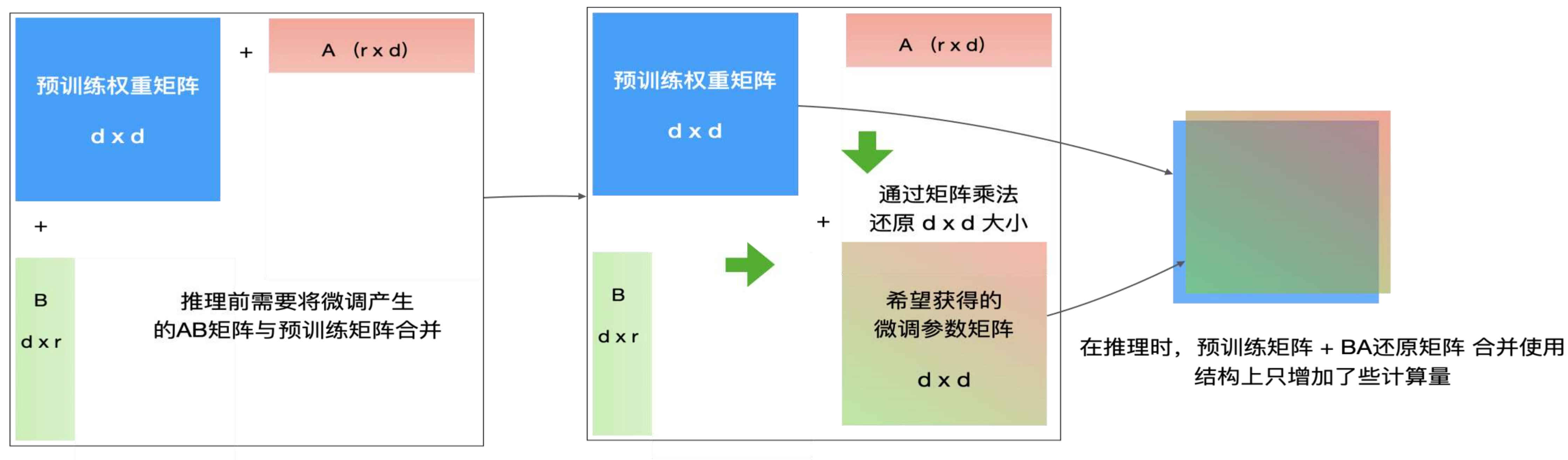
场景决策：

何时用 RAG？
何时用微调？
何时两者结合？

方案	开发成本	推理成本	维护难度	适用阶段
Prompt + RAG	低	低	低	PoC / MVP
Tool Calling	中	中	中	功能扩展
微调（Full）	高	高	高	成熟业务
微调（LoRA）	中	中	中	规模化落地

轻量微调三剑客

- LoRA: 低秩适配, 训练快, 效果好
- P-Tuning v2: 可学习的Prompt, 适合任务特定优化
- Adapter: 插入小模块, 模块化更新



对比维度:

方法	最佳场景	数据量需求	训练效率	模块化能力	典型任务类型
LoRA	快速适配、生成类任务、资源受限	中到大	高	中	文本生成、对话、翻译
P-Tuning v2	小样本、分类/抽取任务、Prompt工程优化	小到中	高	低	分类、NER、意图识别
Adapter	多任务系统、模块化部署、持续学习	中到大	中	高	多任务学习、企业平台集成

适用场景分析

- 数据量少（<1k条） → 优先使用 P-Tuning 或 Prompt Tuning
- 性能要求高、资源充足 → 选择 LoRA，平衡效果与部署成本
- 需模块化升级、多任务切换 → 使用 Adapter，支持热插拔
- 特定领域知识注入（如法律、医疗） → LoRA + RAG 联合使用
- 低延迟服务场景 → 避免 Adapter，优先 LoRA 或 P-Tuning

案例

对外呼对话数据进行意图识别。

输入一段对话，模型会给出该对话的意图分类标签

（{'企业生产': 1, '催收催缴': 2, '营销': 0, '通知': 4, '验证码': 3}）以及相应的概率。

https://www.modelscope.cn/models/iic/nlp_structbert_outbound-intention_chinese-tiny

09 AI系统工程化实践（生产部署）



落地层 如何进行生产部署

系统架构模式的复杂性

系统架构模式：构建模块化、可组合的AI系统骨架

- 选择适合业务复杂度的检索架构，Modular RAG vs. Graph RAG
- 仿照企业组织构建AI协作体系，分层Agent团队架构设计
- 单一技术无法满足真实场景需求，RAG + Agent + Fine-tuning 混合

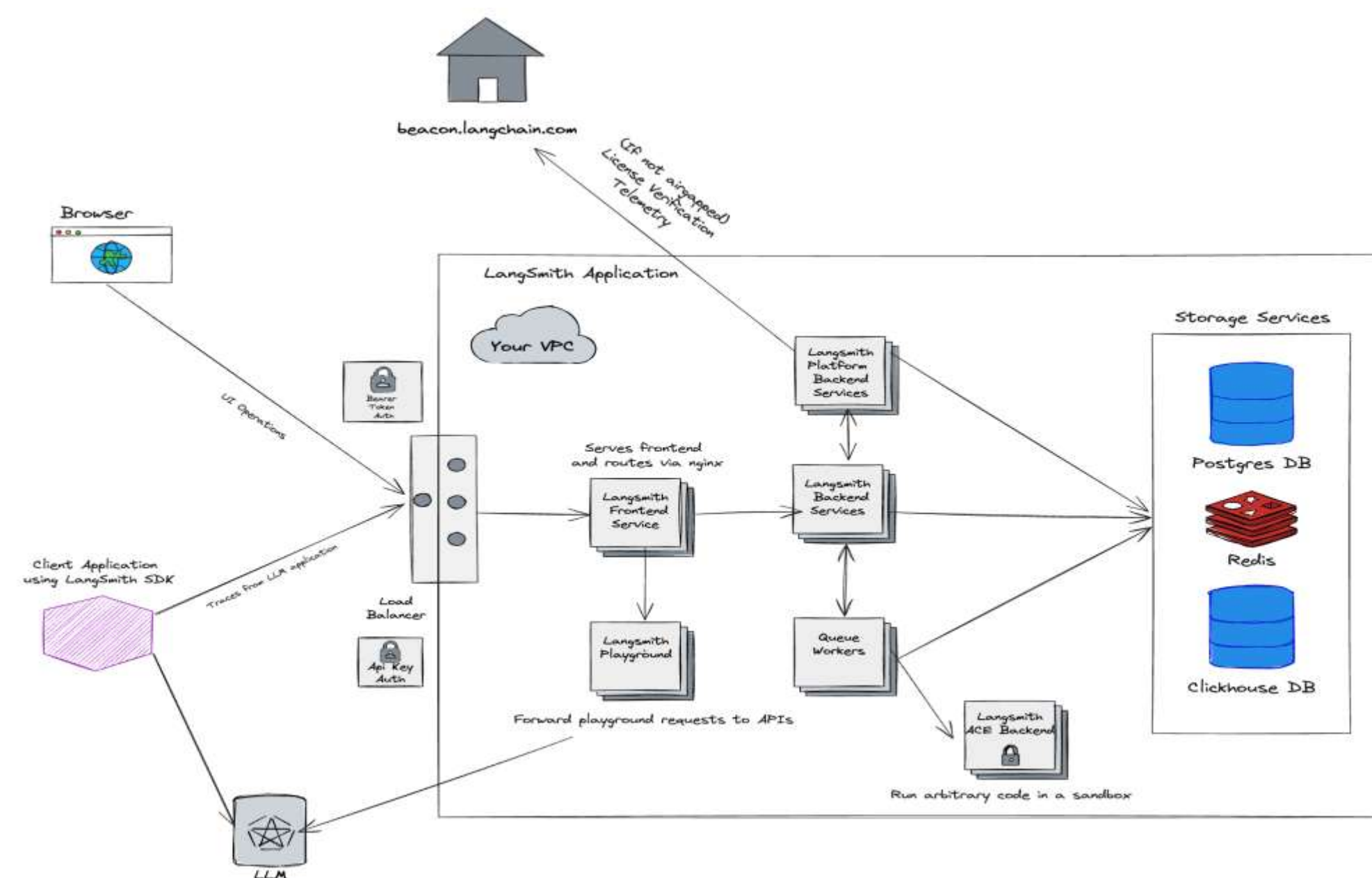
系统设计

可观测性与调试 (Observability)

日志与追踪：使用 LangSmith、OpenInference、W&B 追踪每一步调用

评估 (Evaluation)：建立输出质量评估指标（正确性、一致性、工具调用准确率）

Prompt 版本管理与回滚



图片参考: https://langsmith.langchain.ac.cn/self_hosting/architectural_overview

安全性与合规 (Security & Compliance):

- 敏感信息过滤 (PII Detection)
- 提示注入攻击防御
- 权限控制 (不同Agent可调用工具集)
- 内容审核机制

性能与部署 (Performance & Deployment):

- 推理加速: vLLM、KV Cache 优化
- 缓存机制: Redis 缓存相似查询
- 高可用部署: Kubernetes + FastAPI
- 成本控制: Token监控、小模型兜底策略
- CI/CD for AI: 自动化测试、灰度发布

项目稳定性保障策略

- 防止无限递归或死循环：
 - 设置最大循环次数 (max_turns)
 - 引入“对话熵”检测重复行为
 - 使用 Plan-and-Execute 模式避免反复尝试
- 可视化 Agent 交互过程：
 - 使用 LangSmith 或自研 Dashboard 展示消息流
 - 记录 sender → receiver → content → tool_call 全链路
 - 支持时间轴回放与错误定位

总结

维度	传统 AI 开发	工程化 AI 系统
关注点	模型性能	系统稳定性
架构	单一Pipeline	分层、模块化
调试	手动打印	全链路追踪
安全	忽视	默认安全设计
发布	手动部署	CI/CD自动化
成本	不敏感	精细化管控

项目一：多任务问答助手（LangChain）

类型	内容
输入	用户自然语言提问（支持多轮对话）示例： “今天北京天气怎么样？” “最近有哪些科技新闻？” “昨天说的那条新闻是谁发布的？”（依赖上下文）
输出	- 准确、结构化的自然语言答案 - 明确列出本次响应所使用的工具（如 WeatherAPI, NewsAPI） - 支持 JSON 或文本格式输出，便于前端集成

关键挑战：

- 如何判断是否需要调用工具？
- 如何防止无限递归或死循环？
- 如何记录对话状态？

技术栈参考：

LangChain + Python
FastAPI（提供REST接口）
Redis（缓存 & 会话存储）
vLLM 或 HuggingFace LLM（本地/云端大模型）
OpenTelemetry / LangSmith（追踪与调试）
Docker（容器化部署）

企业客服降本增效助手

背景：
某电商客服系统：用户提问“我要退货”时，30%因方言/错别字导致人工介入，日均损失\$800

请你设计业务决策看板？

Q1为什么值得上线？ 方言提问准确率从42%→89%
工程决策证据链 ？
验证项 要求 实测 证明方式
工具调用必要性 方言准确率>85% 89% bad_case.txt

Q2 成本收益计算？	旧的	新的	日收益
指标： 人工介入率	30%	28%	2%

项目二：多Agent协同客服系统（基于 Autogen 的工程化实践）

- 项目目标：构建一个具备任务分工、自主协作、错误容错与可观测性的智能客服系统
- 在真实企业场景中，单一AI助手难以应对复杂的客户服务流程（如订单查询、退换货处理、投诉升级等）。
- 本项目通过构建多角色Agent团队，模拟真实客服组织架构，实现任务自动分发、协作处理与闭环管理。
- 可以 Dify、Coze workflow 来实现

总结

AI 工程化课程路线图



THANKS

 极客时间 | 训练营